School of Electronic Engineering
and Computer Science

Final Report

Programme of study:
Electronic and Electrical Engineering

Project Title:
Medical Imaging using Optical Coherence Tomography

Supervisor:
Dr Rob Donann
Dr Peter Tomlisons

Student Name:
Hanya Ahmed
181014636

Final Year
Undergraduate Project 2019/20

Queen Mary
University of London

Date: 11 May 2020

Abstract

According to the World Health Organization, in 2016 a study occurred where it estimated that 3.58 billion people were affected with tooth decay (World Health Organization, 2016).Tooth decay is known to occur when a large amount of sugary, sticky foods is consumed. X-rays can be used to examine the teeth and detect tooth decay. Due to its high penetration depth, X-rays can observe the full depth of the tooth. However, it uses harmful ionization radiation that can lead to serious illnesses. Therefore, this project aims to use Optical Coherence Tomography (OCT) as an alternative. This is an arising, non-invasive medical imaging method. It is known to use low coherence light to acquire the images. This achieves a higher resolution than ultrasound imaging, but a lower penetration depth. However, several disadvantages arise due to the use of low coherence light.

The first drawback is that it introduces noise to the OCT images, which produces a difficulty for the dentist to correctly read the image and give a diagnosis. Another drawback occurs due to the low penetration depth. OCT provides 2D images when the clinician requires a 3D model to be able to detect the depth of tooth decay appropriately. A further disadvantage is the inability to compare other medical images to OCT images.

Hence, the aim of this report is to use OCT images to detect tooth decay. This has to be done in several steps. This report achieves successful denoising, registering and reconstruction of OCT images to aide in the detection of tooth decay *via* Machine learning (ML) and modelling. Denoising was achieved through the usage of multiple filters that were inputted into the convolution neural network (CNN) and further connecting a second CNN to mask the images to identify all important edges. Registering OCT images to XMT images was reached through experimenting between Multimodal and Monomodal registering. Multimodal affine registration was the leading technique with that outdone the rest. Reconstruction was successful during this project, however using ML to fully detect early decay was not due to time constraints and COVID-19.

Acknowledgments

Firstly, I would like to thank and show my appreciation to my project supervisors; Dr. Robert Donnan and Dr. Peter Tomlins. As well as my personal advisor, Dr. Tijana Timotijevic. All who have been supportive and invaluable throughout this year. The assistance provided was greatly appreciated.

I wish to extend a special thanks to my parents and my sister for the constant love, inspiration and support throughout. As well as my friends for continuous solace and motivation.

Contents

Chapter 1: **Introduction**

Medical imaging is an approach of capturing images of the human body internally. Medical imaging was accidentally discovered in 1875 through a number of researchers observing cathode rays, which produced the first means of X-ray. Due to the usage of electromagnetic radiation, this technique is invasive and releases harmful radiation to human beings specifically children. In 1991, Optical Coherence Tomography (OCT) later was introduced (Huang D., 1991). OCT utilizes low-coherence light to capture micrometre resolution, where it typically operates using near-infrared light. Therefore, it is a non-invasive technique. Although medical imaging and specifically OCT are used in several fields within the health industry, this report will focus on the dental field. One of the major complications in the dental field is tooth decay. It results from the destruction of enamel occurring by acid producing bacteria on the tooth surface. This is more predominant in children due to increased sweet consumption. Immature tooth decay would reside in between teeth; thus a dentist would not be able to confirm decay without an extraction of the tooth for visibility. Although OCT has several advantages that will be discussed further in this report, current practices cannot detect tooth decay due to limited visibility.

To produce effective 2D OCT images to detect tooth decay, Machine Learning (ML) is introduced. ML is a subset of Artificial Intelligence (AI) where it is an algorithm that has the capability to learn and clarify pervious data, without the user interfering. In addition, it can develop a program through observing, learning new data, instructions and patterns. Through the learning, it prepares for future new data where better decisions will be made. In this project, a 3D image will be constructed through the denoising of 2D images and reconstruction. Where the 3D image will be processed through a deep learning algorithm to detect early stages of tooth decay in between the teeth.

1.1 Motivation

The U.S Department of Health and Human Services have deduced that 18.6% of American children and 31.6% adults have untreated caries and untreated tooth decay (U.S Department of Health and Human Services, 2014). According to Public Health England, 9 out of 10 children required tooth extractions due to tooth decay. However, it could have been avoided (Michaela Goodwin, 2018). Across the years, various medical fields used OCT imaging to detect early carious lesions, properties of dental materials, micro-fractures, pulpal inflammation, premature inflammatory changes in the periodontal tissues and initial dysplastic changes in oral malignancies, etc. However, there are still no procedures that are adequate to detect tooth decay, which could be very pivotal. This would lead to a decrease of tooth extractions along with number of X-rays required. Furthermore, prevention of severe cases could occur through early detection where the dentist would advise the patient to change their diet or extra flossing. Therefore, this project incentive is to enable early tooth decay detection in a non-invasive manner.

1.2 Problem Statement

Researchers applied OCT imaging to multiple areas in the medical field for detection reasons. However, it has not been implemented successfully in dentistry yet, especially for early tooth decay detection. Therefore, a 3D model will be constructed to display tooth decay to the clinician with the usage of 2D OCT images.

1.3 Objectives

The targets this project aims to achieve are:

- Maximize the removal of noise in OCT 2D images through MATLAB.
- Expanding the MATLAB code to reconstruct 2D OCT images to form 3D images.
- Further expansion to register the OCT images to XMT images.
- Enhance the code by connecting Python to MATLAB for further detection of tooth decay.
- Link all pieces of code to produce a web application, as well as a software through Django and Tkinter respectively.

1.4 Report Structure

Within this report, the next chapter will include background research and what was concluded from it. Chapter 3 will contain the methodology and results that were obtained, where Chapter 4 will include a few snippets of code to show the implementation. Finally, Chapter 5 will analyse the results, reach a conclusion, as well as investigate further work that can be considered.

Chapter 2: **Literature Review**

This chapter describes research that has been done and is applicable to the different elements enforced in this project. This includes a brief overview of dentistry, imaging techniques, OCT, ML and programming languages. It also outlines how the respective methodology in Chapter 3 was finalized, through researching the different approaches with their respective advantages and disadvantages.

2.1 Dentistry

Teeth are sectioned and termed in a way called “Dental Jargon” (Wilkinson, 2017) (Figure 1). Teeth incorporate many layers; enamel is the harder outermost layer (American Dental Association, 2019) which acts as a protection layer. Yet with certain diets, such as high intake of sugar leads to an increase of bacteria in the mouth. This weakens the enamel and leaves the tooth vulnerable to pits, cavity or carries (Oral-B, 2019).

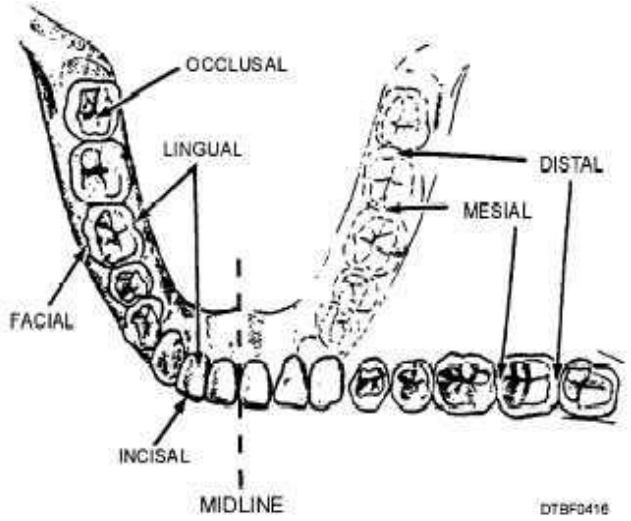

Figure 1 : Diagram that shows a part of the human mouth to show how the teeth are sectioned through the Dental Jargon (Navedtra, 1999). Occlusal describes the “biting surface of the molar and premolar teeth”. Lingual describes the “inside edge of the tooth”. Incisal describes the “biting edge of the incisal and canine teeth”. Mesial describes the “front edge of the tooth”. Distal describes the “back edge of the tooth”. Facial describes the “side of a tooth next to the inside of the lips”. (Wilkinson, 2017)

2.2 Medical Imaging

Over the years, there are many different approaches of imaging in the medical field, however this project concentrates on the dental field. X-rays are most used where low levels of radiation, to show the interior of the mouth (Brian Krans, 2018). Bitewing, periapical, panoramic X-rays are the 3 different types that are mostly used (Davis, 2017). Bitewing catches early cavities, periapical shows the whole tooth from crown to root and panoramic shows the entire set of teeth (Davis, 2017). As mentioned above (1.1), X-rays emit radiation leading to health risks, such as cancer. However, there had been no reports connecting X-ray exposure to those health risks (Su-Yeon Hwang, 2018).

Computerised Tomography (CT) and Magnetic Resonance Images (MRI) are other dental imaging techniques. They are used for specific applications such as; when dental implants are needed or to evaluate the tissue of the patient (Davis, 2017). Their downfall is that they are extremely expensive. A cheaper imaging approach is ultrasound, where

it employs sound waves to assess hard tissue in dentistry. However, its penetration depth is 150 micrometres (Demirturk Kocasarac H, 2018). This is shorter than MRI and CT (Figure 2). An emerging new imaging technology is Optical Coherence Tomography (OCT), it has a leading resolution but a limited penetration depth (Figure 2). Considering the shorter penetration depth, OCT and ultrasound are capable of occurring under motion unlike MRI and CT. This permits the ability of portability (Colston B.W., 1998). OCT uses low-coherence infrared rays to harmlessly penetrate the gums (Yu-Chi Lai, 2019). OCT is allowed for a longer exposure time, since it is biologically safe at a certain level (Colston B.W., 1998). Hence, OCT is non-invasive, unlike X-rays and it has a lower cost when compared to MRI and CT. In addition, due to its high resolution OCT is the best imaging technique for implementation in the dental field (Karimian, et al., 2018).

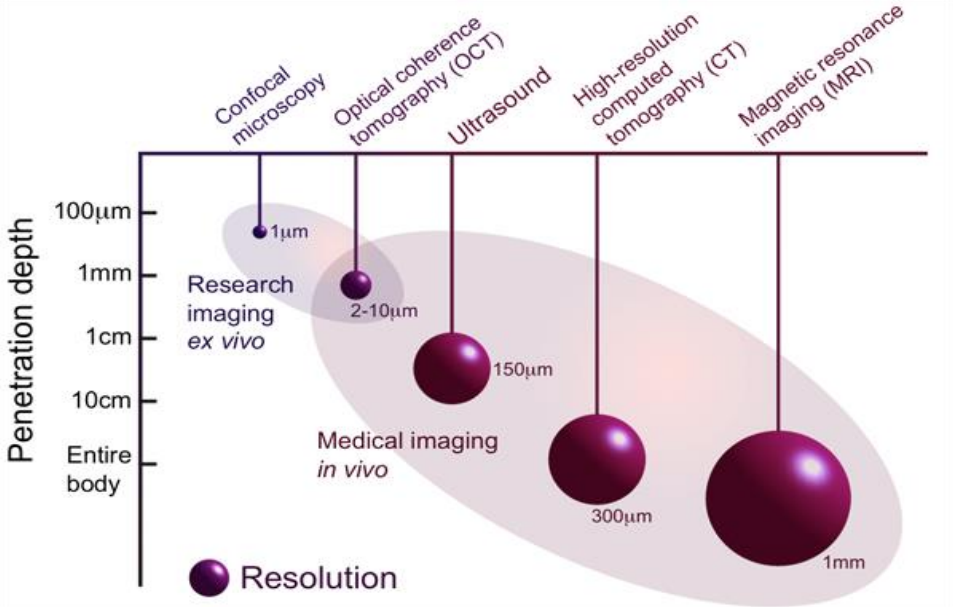

Figure 2: This graph shows the relation between the resolution and penetration depth for all the imaging techniques mentioned in the paragraph above (Colston B.W., 1998).

2.3 Optical Coherence Tomography

OCT can be classified into two domains: time-domain and frequency-domain OCT. Frequency-domain OCT can further be divided into: spectral-domain OCT and swept-source OCT.

2.3.1 How it works:

- OCT uses coherence which is a characteristic of light. Light is said to be coherent when all the rays within maintain a fixed and calculated phase over a period of time.
- Noise is referred to unsought addition to images, that changes the pixel's intensity therefore it transforms the image to an unoriginal image (Chen, 2013). Low-coherence is said to be caused when noise is introduced to the process, thus it gives rise to a poor signal to noise ratio which results in disturbances in the OCT images (Yu-Chi Lai, 2019). There are many types of noise, however the main type of noise correlated to low-coherence light is speckle noise (Wong, 2010),
- An optical probe aims a low-coherent light to the sample, where it penetrates the samples' surface, which waits for the reflected light to rebound. Afterwards, it sends those signals to the interferometer (interferometric detector) through an optical fibre for analysis (NOVACAM, 2019). The schematic diagram is shown in Figure 3.

- An A-Scan is taken by the interferometer from the clarification of optical data from each single scan point as an interference pattern and it documents it as a depth profile. On the other hand, a B-Scan can be taken by replacing the probe in a linear fashion covering the sample which shows a cross section of the tooth (NOVACAM, 2019).
- The scanning occurs successively, so both the emitted and reflected light advance on the same axis.
- The data collected by the Michelson interferometer is sent to a computer to produce an A-scan or B-scan, through inputting the interference spectrums into a Fourier transform method.

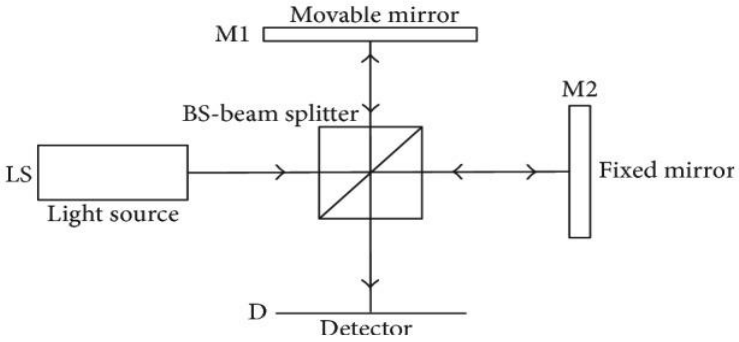

Figure 3: A schematic diagram that displays the setup of Michelson interferometer in the OCT (Monika Machoy, 2017). The light source (LS) emits a light wave that is split into two beams through the beam splitter (BS), one perpendicular and one straight beam. The perpendicular beam is reflected off the movable mirror (M1) and through the beam splitter again straight to the detector. The straight beam is reflected off the fixed mirror (M2) and through beam splitter to the detector (D). The beam incident in the detector creates an interference image (Monika Machoy, 2017).

2.4 Machine Learning

Machine learning (ML) is a partition of computer science and AI that builds and trains programs using past data to predict the future. The whole concept of ML is to use pattern recognition so the computer can acquire data without being programmed precisely for specific tasks or with human interaction (Brownlee, 2016). ML can further be split into two subsets: supervised and unsupervised learning.

2.4.1 Supervised Learning

In this subset of ML, a labelled training set is used. This is a number of instances where their attributes are so far known. It is used to teach the algorithms using input-output pairs established. The algorithm may be given a new instance with an unknown attribute. In this case, the aim of the program is to approximate the value of the unknown attribute. This is done using the collection of previous known pairs (Brownlee, 2016). In this sense, the scheme operates under supervision by the provision of actual outcomes for training examples (Witten, 2017). The algorithm stops when it accomplishes a justifiable outcome. Supervised Learning can be further split into two classes; classification and regression. Where classification focuses on categorical data and regression on continuous data. Categorical data is when data can be divided into discrete classes. Continuous data is when the variable is a real number. The main examples of classification are; neural networks and decision trees. Where, examples of regression are; linear regression, nonlinear regression and decision trees.

2.4.2 Unsupervised Learning

On the other hand, unsupervised learning is used for problems where instances lack historical labels (Brownlee, 2016). Thus, the algorithm must find out the underlying

structure or the distribution of the dataset. This should lead to the identification of anomalies and outliers, as well as reducing processing time. The system then is left to understand the data and present the system of instance in a structure (Fritzke, 1994). Unsupervised learning can be classified into two groups: clustering and dimensionality reduction. Clustering is used when several data points are grouped together due to their similar nature. Whereas, dimensionality reduction is used when there is a large data set that requires to be grouped into a reduced set of attributes by acquiring a set of principle variables.

2.4.3 Convolution Neural Networks

Amongst deep learning techniques, convolution Neural Networks (CNN) is tremendously used for pattern recognition, filtering and denoising especially in the imaging field. The human brain is able to do these activities at a rapid speed. Thus, with the utilization of filters and many layers, CNN has the ability to mimic the human brain (Kocaleva et al., 2016). The technique takes in data, usually images, and trains and tests the network by passing it through several layers. Each of the filters conveyed as layers are recognized as feature identifiers. Feature identifiers are attributes that help identify the label of the image. The convolution layer is the basis of CNN, where it uses the filter and weights from the trained CNN and passes it across the image, hence leads to the production of activation map. Convolution acts like a filter also called neuron or kernel. It is crucial for the convolution's depth to be the same as the input (Karimian, et al., 2018). The filters included in the convolution layer can have numerous different operations such as; edge detection, deblur, sharpening, etc. (Prabhu, 2018). The layers are connected through the neurons of the previous layers, which pass the activation map of the previous convolution. This acts as the input for the next layer. As you go through convolution layers, the features increase in complexity as represented by their activation maps. An array of parameter (filters and weights) is multiplied through the data (images) provided, the receptive field, in CNN. Once all positions and outputs are obtained, an activation map is produced. Those are concluded from covering a larger receptive field. The first few layers are convolution, followed by ReLU and pooling ending with a fully connected layer as well as a regression layer. ReLU layer consist of equation 1, which absolutes the activation maps thus it sets any value below zero to zero (Fu, 2019).

$$f(x) = max(0, x)$$

Pooling layers are used to minimize the number of attributes when the images/data are too large. The process includes down sampling the information where it reduces the complexity of it but retain the important data (Prabhu, 2018). There are different types of pooling: max pooling, average pooling and sum pooling (Karn, 2016). All the activation and feature maps are connected through the process of the fully connected layer. The regression layer outputs a predicted trained response to the input.

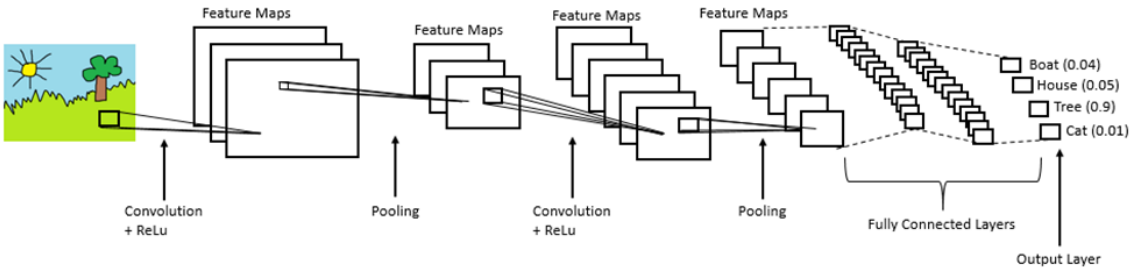

Figure 4:Displays an example of a fully built CNN (Convolution Neural Networks). It shows the convolution layers, their outputs as feature maps after going through an ReLU layer. Afterwards the feature maps go through a pooling layer which decreases the feature maps and then the output is put through the whole process again till each pixel in the image has been processed (Prabhu, 2018)

After the network is built as shown in Figure 4, it is trained so it is able to classify the data inputted later on. The CNN is trained through changing the weights and filters in the convolution layers. The algorithm adjusts the weights by using a process called

backpropagation. Principally, it trains the system till the CNN would give the correct output to certain image features. Backpropagation is split into four components: the forward pass, the loss function, the backward pass, and the weight update. The whole process of backpropagation is one iteration. For the algorithm to be able to run without supervision, it will repeat the procedure for a number of iterations for each batch of images/data (Deshpande, 2017). There are also different optimizer algorithms used to train a system including, Stochastic Gradient Decedent with Momentum (SGDM) and Adaptive Movement Estimation (ADAM). They use mathematical functions to change the weights which are shown in the Table 1.

Table 1: Shows the optimizers and how they change the weights depending on their equation

Optimizer	Equation
SGDM	*Equation 1* $\theta = \theta - n \cdot \nabla_\theta J(\theta; x^i; y^i)$ Where θ are the weights, x is the training example, y is the image observed, n is the learning rate (Ruder, 2016)
ADAM	*Equation 2* $\theta_{t+1} = \theta_t - \frac{n}{\sqrt{v_t + \epsilon}} m_t$ Where θ are the weights, n is the leaning rate and the uncentered and mean variance are v_t and m_t, respectively. (Ruder, 2016)

2.4.4. Error Calculations

In ML the concept of perfecting the algorithm has yet to be accomplished, since errors always occur whilst running programs. There are different ways to test the system and calculations to use that give the user an error percentage to consider. The quality of the program can be tested by the goodness of fit, loss function or error functions. The most popular ones are; Mean Square Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), confusion matrix and loss function. The probable functions used for CNN and classification are MSE, Loss function, and Confusion Matrix. Those are further explained in Table 2. The confusion matrix manifests the number of incorrectly and correctly classified instances for each known class.

Table 2: Shows the error calculations conducted in this project

Error Calculation	Equation
MSE	*Equation 3* $\frac{1}{n}\sum_{1}^{n}(x_i - \hat{x_i})^2$ Where n is the total number of instances, x_i is the known values, $\hat{x_i}$ is the predicted values. (Saxena, 2013)
RMSE	*Equation 4* $\sqrt{\frac{MSE}{N}}$ Where N is total number of instances.
Loss function	*Equation 5* $\frac{1}{2}\sum_{i=1}^{R}\frac{t_i - y_i}{R}$ Where R is the number or response, y_i is the network's estimate for the input, t_i is the output. (MathWorks United Kingdom, 2019)

2.5 MATLAB

MATLAB is an abbreviation of MATrix LABoratory, precisely because it allows matrix manipulations, plotting graphs and data. It is a program that provides access to a high-performance language and an integrated desktop environment (IDE). The IDE is separated into a workspace, command window and an editor. The popular applications of MATLAB include, mathematics, data analysis, modelling and simulation. An important feature included in MATLAB is accenting toolboxes. Those are a family of application specific solutions that allow the user to learn and thus implement. In addition, MATLAB includes a broad number of libraries for an assortment of operations. They are mostly user developed and are available for utilization through MathWorks.

2.5.1 Image Processing Toolbox

An image can be processed on MATLAB using the Image Processing Toolbox. The toolbox includes a comprehensive set of reference-standard algorithms and workflow applications for image processing. This includes algorithm development, visualization and analysis. It enables image segmentation, registration and enhancement. In addition, it can perform noise reduction, geometric transformations and 3-D image processing (MathWorks, 2019).

The process of importing an image into MATLAB involves storing the image into a matrix. Each element in the matrix is a single pixel, where the dimension of the matrix will depend on the images' dimensions. An element can be selected from the matrix; this is called indexing (MathWorks, 2018). To access subsets of elements from the matrix, vectorization is introduced. This leads to a reduction in processing times and errors as opposed to using program loops such as For loops (MathWorks, 2018).

2.5.2 Image Reconstruction

Image Reconstruction techniques are used to form 2D and 3D images from a collection of 1D images. The foundation of these techniques is the Radon Transform, inverse Randon Transform, and the projection slice theorem. For computational methods filtered back-projection and repetitive methods are used (MathWorks, 2019). Image reconstruction is widespread and useful in the medical field as well as the research field.

2.5.3 Image Registration

One of the features of the Image Processing Toolbox, is Image Registration. Image Registration is usually the initial step in the processing of images. It involves the process of aligning two or more different images. It allows the user the ability to compare common characteristics in different images and it helps extract information from the output that would not be available through a single image (MathWorks , 2019). There are four registration methods offered through the Image Processing Toolbox and Computer Vision Toolbox: Control Point Registration, Intensity-Based Automatic Image Registration, Registration Estimator App and Automated Feature Detection and Extraction (MathWorks, 2019). Image registration is often used in medical and satellite imagery.

- Registration Estimator App allows the user to register the 2D images in a precise way, where you can control the tune and registration techniques. It outputs the registered image, transformation matrix and measures the quality of the image.
- Control Point Registration enables the user to physically choose the common characteristics. It is mostly used when automated feature matching is unclear with repeated patterns, or the prioritization of certain alignments depending on the feature.

- Automated Feature Detection and Matching detects characteristics in the fixed images and matches equivalent feature on the moving images, therefore it estimates a transform matrix to align all images (MathWorks, 2019).
- Intensity-Based Automatic Image registration aligns the images using their comparable intensity patterns. A fact that makes this method stand out than the rest is that it is the only technique that registers both 2D and 3D images. This approach is mostly practical for when a large set of images is being registered or automated registration (MathWorks, 2019). Hence, this is the one used to proceed with this project. Within the functions used to register images, there are different types of geometric transforms to be applied (Mathworks, 2019). They are shown in Table 3.

Table 3: Shows the types of Intensity based registering and their functions

Type	Description
Translation	Geometric function where each point of the object must be moved in the same direction and distance (Study.com, 2020). It uses (*x, y*) and (*x, y, z*) for 2D and 3D images respectively (Mathworks, 2019).
Rigid	It consists of the usage of both translation and rotation.
Similarity	It utilizes translation, rotation and scale. It's a non-reflective similarity transformation.
Affine	It consists of the usage of translation, rotation, scale and shear.

2.5.4 Denoising

The simplest way to segment images is to use thresholding. There are numerous methods for thresholding, however the straightforward method involves setting a fixed constant initially. Then checking if each pixels' intensity is higher or lower than the constant, afterwards replace it with a white or black pixel respectively. It simply removes the background of an image, so when the image undergoes processing it focuses on the foreground.

A follow-up of this is denoising. This is a procedure used to remove the noise signals from signals/images. The noisy data is caused by external disturbances. Noise has a negative impact in medical imaging, because it blurs and sabotages the images' quality. Hence denoising is crucial in the pre-processing stage. There are several ways to remove noise; Image Averaging, Averaging Filtering, Wiener Filtering, CNN (McAndrew, n.d.). Those are explained below. Image qualities can be measured depending on a few factors, that are shown in the Table 4.

Table 4: Shows the equations used to measure image quality

Metric	Equation
PSNR (Peak Signal to Noise Ratio)	*Equation 6* $10\log_{10}\left(\frac{R^2}{MSE}\right)$ Where MSE is determined through equation 3, and R is the number of maximum variations between two pixels in the image. (Saxena, 2013)
SSIM	*Equation 7* $\frac{(2\sigma_{xy+}\, c_2)(2\mu_x\mu_y + c_1)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$ This calculates the image similarity between two images through the usage of the weights. Where x and y are the two images that are being compared. (Renieblas, 2017)

- Image averaging is used when there are many different images of the same object, but with different degrees of distortion. Firstly, different versions of the image are made with Gaussian noise, after that the average of all the images is taken (McAndrew, n.d.).
- Averaging filtering is a method that reduces the noise significantly but blurs the image as well. Therefore, if the Gaussian noise has a mean of zero, then the average would filter the noise to zero too, however there will be a smeary nature to the image (McAndrew, n.d.).
- Wiener Filtering uses the local image variance as its main component; therefore if the variance is high the filter would not smooth the picture as much. Whereas if the variance is low, the smoothing effect would be higher (McAndrew, n.d.). Wiener Filtering is used as a fundamental foundation for the widely used BM3D image restoration method (Ymir Mäkinen, 2014).
- As mentioned above, CNN displays powerful capacity with deep architecture where it is used to deeply analyse the data sets. Also due to the strong ability to learn, CNN has become the most successful technique to solve the problems for image denoising (Kai Zhang, 2016). It also uses external denoisers (i.e. BM3D, CBM3D, etc.) as layers to increase the efficiency and performance. It also eliminates the long processing timings (Tanzila Saba, 2010).

2.6 Python

Python is a widely common programming language. This is due to the fact that it runs on most platforms and the large number of libraries it acquires. The libraries contain a vast variety of functions that would allow a large number of applications for the user. In addition, it uses a general-purpose language with easy to read syntax. This makes it one of the most user-friendly programming languages, since it works closely with Graphical User Interface (GUI) toolkits. The GUIs help the user adapt to the method of writing code in Python. There are many toolkits and libraries in python for GUI programing, however the most common one is Tkinter (Python, 2019). Python is relatively superior when compared to other programming languages, in terms of data manipulation and repeated tasks for scientific and engineering codes. It also has a web framework that supports fast development and sensible design called Django. It is widely known for its easy concept, security and scalability (Django Software Foundation, 2020). It is built solely to make web development trouble-free for unexperienced developers. Python will be used in this project through the usage of the ML libraries such as Scikit-learn, Scipy, etc. Due to the particular reason of it being one of the most adaptable programming languages (Lin, 2012). Python will also be used to create a functional web application and software through Django, Tkinter respectively. This will make it easier for clinicians to operate with.

2.7 What has been done before

Optical Coherence Tomography was introduced to the medical field in 1991, to image the human eye, and till this day OCT has the largest footprint in ophthalmology (Wang Y., 2008). In 1998, a few researchers from University of Connecticut and Laboratory of Medical Technology of Livermore, they sought out a prototype OCT in dentistry (Colston B. W., 1998). In the same year Feldchtien et al. directed experiments to check the usefulness of OCT in the detection of lesions in both hard and soft structures of the oral cavity.

Two years later, the same laboratory experimented different wavelengths of light for OCT. It was concluded that the implementation of longer wavelengths increased

effectiveness (Otis L. L., 2000). In 2005, some dentists were trained on OCT and were able to inspect OCT scans and find fissure sealants, composite fillings or tissue enamel (Otis L. L., 2003) (Holtzman J. S., 2010). After that, researchers proved there is an advantage of using OCT to detect caries in both primary and secondary teeth. It can effectively diagnose early tooth decay in dentistry (Maia A. M., 2010).

Although OCT images have been processed, using different algorithms, throughout the decade in the medical field; CNN has only been recently utilized to detect certain illnesses. CNNs were being used for image character recognition in 1988 by W. Zhang et al. (Zhang, 1988), afterwards in 1991, the algorithm was changed so it can be applied to medical image processing (Zhang, 1991) and automatic exposure of breast cancer in mammograms (Zhang, 1994). Subsequently, it has been used to classify brain tumour segmentation (S.Pereira, 2016), cerebral micro bleeds detection (QiDou, 2016), then analyse images of skin (GoodpasterBH, 2001). They were recently widely used in the dental field to detect early dental caries (Karimian, et al., 2018) structural cracks and split lines (Monika Machoy, 2017).

One of the main challenges encountered in image analysis has been noise. Therefore, denoising was introduced to restore images. Numerous methods were used to restore images throughout past few decades. The “state-of-the-art” (K. Dabov, 2007) methods are BM3D, Nonlocally Centralized Sparse Representation (NCSR) (W. Dong, 2013).The most known and used is BM3D method for image debluring (Ymir Mäkinen, 2014) (Katkovnik., 2015), however for single image super resolution (SISR) CBM3D was presented (Katkovnik., 2015). CBM3D is a subset of BM3D. Afterwards a similar idea was presented for denoising, deblurring and inpainting called half quadratic splitting (HQS) (F. Heide, 2014). A few recent articles have combined BM3D with CNN to increase performance and speed of denoising images. The accuracy results that has been accomplished are quite reasonable (Kai Zhang, 2016) (Tanzila Saba, 2010) (Lebrun, 2012) (Hasan, 2019) (Chong, 2013).

2.8 Summary

Considering the literature review, it is evident that OCT is an important emerging research tool in the medical research field. This is due to advantages it acquires such as (K. Dabov, 2007):

- high resolution when compared to ultrasound
- non-radiative when compared to X-ray
- lower costs when compared to MRI and CT
- non-invasive

Optical Coherence Tomography has been used in many departments in the medical field for a long period including ophthalmology and neurology. However, it only started evolving in dentistry a few years ago. It has been implemented into hard and soft tissue imaging; detection of oral cancer; periodontal disease and early detection of caries (Filho, 2013).

As mentioned above, there is yet an algorithm to remove all of the noise from OCT images. However, attempting an already challenging problem is time-consuming and ineffective since it is easier for the user to detect the noise visually. Machine learning was also used to attempt denoising OCT images, but it was not perfected. The use of OCT images and deep learning to detect tooth decay in between teeth has not been done.

Hence, this project will use MATLAB to reconstruct the 2D images to model a 3D image. It will go through a CNN which will use external denoisers as convolution layers i.e. BM3D and NCSR. The restored image will then go through another CNN to analyse the 3D

image and detect tooth decay. Error calculations will be carried out during each process to check the credibility of the whole process. Such as PSNR for the denoised images, MSE and confusion matrix for the data analysis, etc.

The main objective of this project is to achieve denoising and modelling of 3D images should aid in achieving the overall aim of this project. This image analysis, upon completion, would assist dentists in detecting tooth decay in early stages. This would be especially crucial in paediatric dentistry, due to their increased sugar consumption when compared to adults.

Chapter 3: **Methodology**

Considering the literature review in Chapter 2 and previous research an approach for this project was deduced. This chapter portrays the methodology that will be implemented to achieve the projects' aims and objectives. The method can be divided into five essential parts (Figure 5).

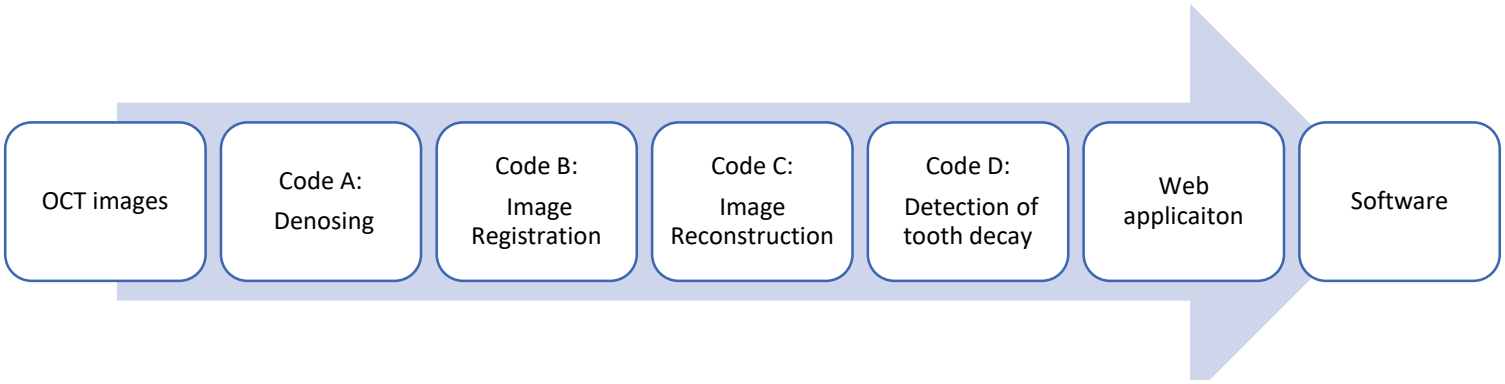

Figure 5: Block Diagram showing the outline of the methodology. Each block is described in detail in Section 3.1. Starting from OCT images to Software in Sections 3.1.1 – 3.1.7 respectively.

3.1 Design

3.1.1 OCT Images

The OCT images provided were sampled through, arranging a rotating stage on the SD-OCT scanner, where the tooth is imaged in different angles. This was done in the Whitechapel Dentistry Hospital Lab. This set-up presented two types of images:

- "Theta-stack": where the tooth is rotated 360 degrees and a B-scan is taken at every degree on the x-axis
- "Y-stack": where the tooth is at a fixed angle and B-scans are taken along the y-axis.

In this project, the Y-stack images will be used. However, the codes are tested on many different images so it can generalize. This will allow different clinicians to take various images.

3.1.2 Code A

After collecting the images from Dr Tomlins, the OCT images mostly had speckle noise (Chen, 2013). The noise appears as "grainy white speckles" that occlude the image (Schmitt, 1999). This is an occurrence in medical imaging that hinders the detection of boundaries (W. Dong, 2013). Therefore, this introduces the need to denoise the image before using the 2D OCT images for image reconstruction.

The first code is created to denoise the 2D images through the utilization of MATLAB and its Image Processing Toolbox. The code will contain certain steps as shown in Figure 6:

1. The image is read through the usage of function imread() and changed to grayscale using r2gby() function, since BM3D will only read grayscale images.

2. The image then goes through thresholding to help remove the background. Therefore, the image's foreground becomes clearer. This is done by segmenting the image into two portions, where the first portion includes pixel values higher than the threshold and the other portion includes pixel values less than the threshold. After that one of the portions go through imfuse() where it boosts the contrast.
3. After that Fast local laplacian (FLLP) filtering function where it filters out the image with an aware to the edges (Paris, 2011).
4. The image will then be inputted through the first BM3D. It denoises the image using Wiener Filtering. Then the outputted image will be put through the BM3DDEB.m where the image goes through the process of deblurring. It will output the image at each stage of denoising. The PSNR is also calculated to show the signal to noise ratio. Moreover, the image is sharpened through the use of the function imsharpen().
5. Further on, CNN will be built with many layers using the functions dcnnlayers () where it builds the convolution, batch normalization layer, ReLu, regression layer. Then it will be trained using the ADAM optimizer, verbose to show the training process, and 20 epochs. This is the amount of times the whole training data is run past the whole network. In addition, a learning rate of 0.001 will be used, so it will learn more specifically but quite slowly. Moreover, the network is trained to enable the user to analyse the network using the loss function and RSME.
6. After training the CNN, it is tested with one of the raw images and the output will be the noisy image and noise-free image using the function imshowpair().

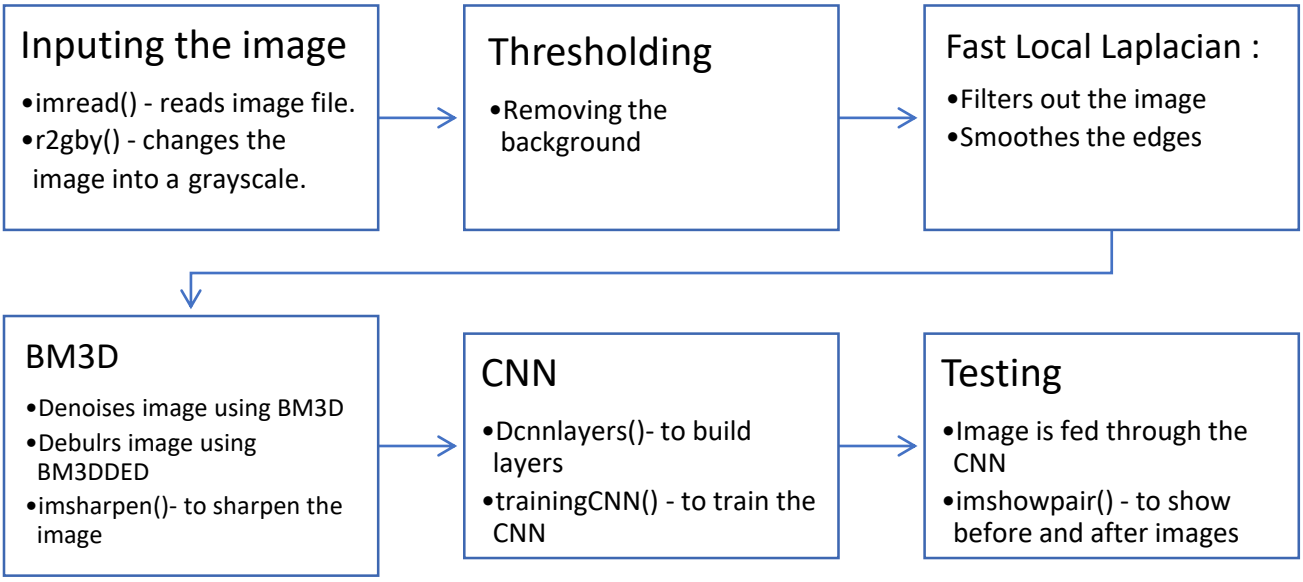

Figure 6: Block Diagram to show the steps taken in Code A. The image was put through thresholding, FLLP to remove the background noise and smoothen the edges respectively. Further through BM3D and CNN to denoise and train the CNN to further denoise testing image. This process was chosen to optimize the removal of noise with training the CNN with the output from each step. Each of the output image serves a purpose for the CNN to learn.

3.1.3 Code B

The second code will be used to implement image registration. This will align OCT image files onto XMT image files. It will be executed using Intensity-based registration due to its ability to handle a large collection of images as well as automated registration. This method is an iterative process, where several features of the images metric, optimizer and transformation type are specified before the process begins. The output will be a measure that determines how similar the images are for the accuracy of the registration

(MathWorks, 2019). The functions used to make the block diagram true are as follows (Figure 7):

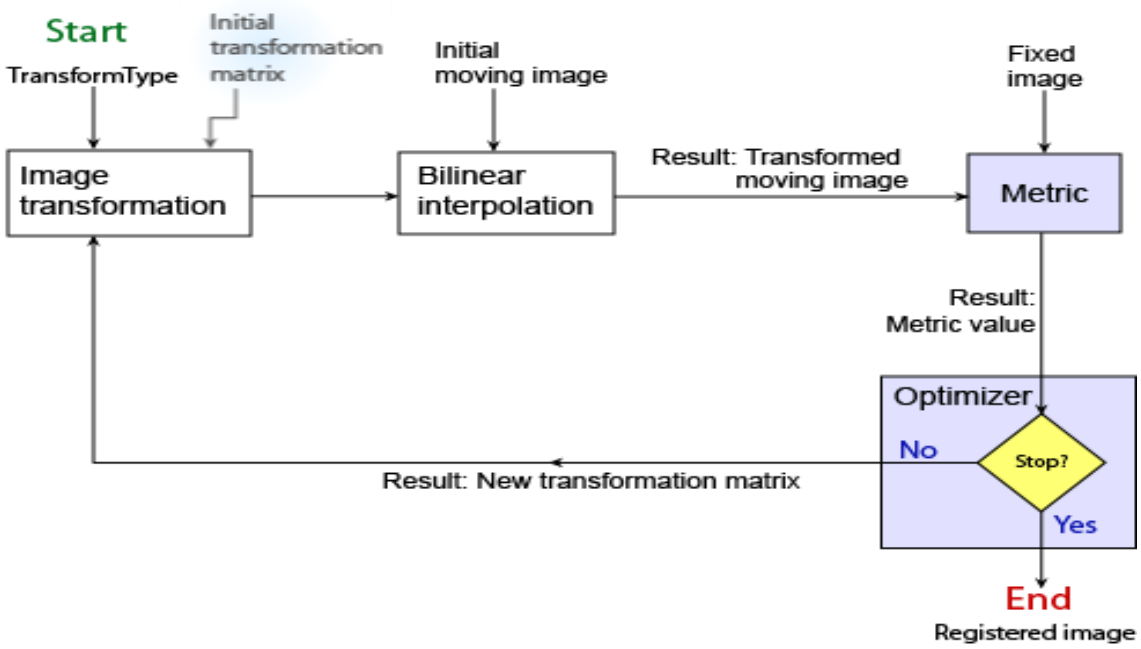

Figure 7: Block Diagram to show the process of Code B.

1. Inputting the images into the workspace using imread().
2. Further on, creating an optimizer and metric using imregconfig(). In this project it will try both Monomodal and Multimodal capture scenario, which they use different metrics and optimizers respectively.
3. Additionally, the images are then registered using the function imregister(). As mentioned in Table 3, there are many types of registering. Therefore, this code will try all various types, to identify the best registration.
4. Finally, the images are shown through the usage of imshowpair().

Monomodal capture scenario uses the Mean square error metric configuration and the Regular step gradient descent optimizer. Multimodal uses Mattes mutual information metric and One plus one evolutionary optimizer (MathWorks, 2019).

3.1.4 Code C

The third code will perform the process involving image reconstruction. The foundation of image reconstruction in MATLAB are the mathematical functions: Radon transform, inverse Radon transform, the projection slice theorem (MathWorks, 2019). The algorithm uses filtered back projection and iterative methods on the theta-stack images to reconstruct the images from 2D to 3D. The Shepp-Logan phantom technique is utilized through the usage of function of phantom() where it outputs an image of a head phantom. The accuracy can be tested using the radon and iradon functions.

3.1.5 Code D

The fourth piece of code will be written in python using the ML library called tensorflow and keras. It utilizes the concept of CNN and unsupervised learning, where the reconstructed 3D model is inputted into the neural network to detect tooth decay. The network will be built through many layers, so it mechanically learns a hierarchy of complex features. The first layer will be an input image layer which will then be inserted into convolution layers (Karimian, et al., 2018). As the process ensues, CNN will be taught whilst going through the layers of the network. After that the network is trained

using certain training options and the trainCNN() function. During the testing period, KNN neighbours will be used to identify the depth of tooth decay within teeth. The output will consist of inspecting the confusion matrix, MSE and the correctly classified instances. There is an intention of applying a heat map to the 3D model of the tooth, to show how far the tooth decay is. This should enable dentists to grasp the concept of the output given to them, therefore it will be more user friendly.

3.1.6 Main Script

This will be the final piece of code, where it will connect all the pervious codes. It will be written in python using matlab.engine library to allow Codes A-C to connect to Code D. It will have minimal graphical user interface, since this is mostly aimed for researchers to observe the results. Meaning, it will prompt the user for the type of image they wish to input. Next, it will prompt for the type of task to perform where will trigger the matlab engine to start and perform the task. The result will then return to python where it'll show the results in a pop-up window using the matplotlib library.

3.1.7 Web App

In this part of the project, a web application will be developed through the usage of Django in python. The objective will be that it contains similar cycle as the main script. However, with more GUI so it gives guidance to users that do not have a computing background. It will display all of the functions possible in a reasonable manner and output the resulting images as well as save them locally.

3.1.8 Software

Furthermore, GUI will be enhanced for users that have little knowledge of computing through developing software *via* the usage of Tkinter library in python. It will closely also follow the same cycle the main script follows. It will also save the resulting images in the local folder of running the software. A user interface design for the software is shown (Figure 8).

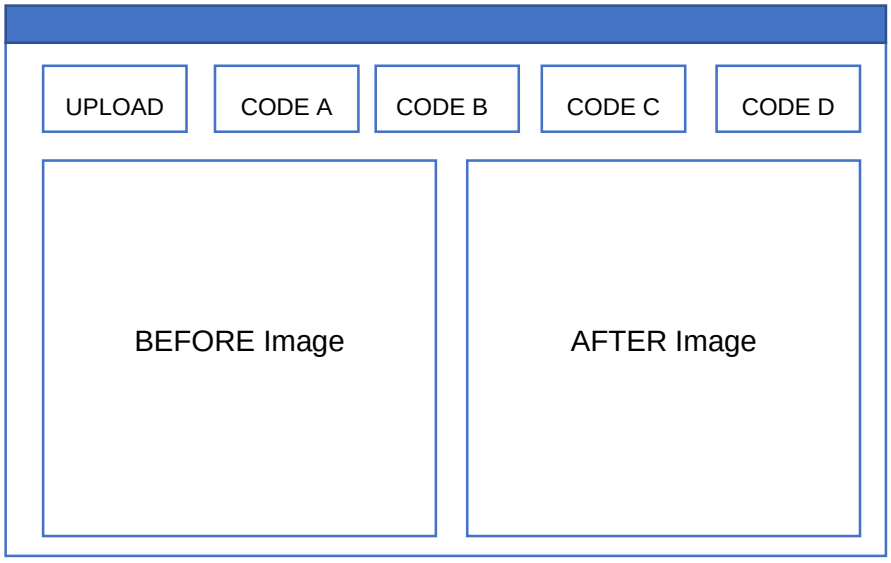

Figure 8: The GUI designed for the software to be built. The top bar is utilized to upload and choose the function the user wants to occur to the before image. The output will be displayed next to the before image.

3.2 Implementation

3.2.1 Code A – Denoising

This code followed the design shown in the Chapter 3.1.2 above.

3.2.2 Code A – Masking

For this code, instead of asking the CNN to denoise and mask the image in the same CNN. It was changed to two separate CNNs where one will denoise the images, the second to mask images. The first CNN will be trained using the training images produced from the thresholding, FLLP, BM3D filters the original image goes through. The second CNN will be further trained using images that go through edge detection depending on the intensity of the image. This occurred through the usage of the edge() function in matlab. Further edge() has various methods of detecting lines that are described in Table 5 (MathWorks, 2019).

Table 5: Explains the numerous methods available through edge function (MathWorks, 2019)

Method	Description
Sobel	Detects edges using the Sobel approximation to the derivative of the points where the gradient of I is at its maximum.
Prewitt	Similar to Sobel, but uses the Prewitt approximation to the derivative.
Roberts	Similar to Sobel, but uses the Roberts approximation to the derivative.
Log	Filters the image with a Laplacian of Gaussian Filter, then detects the edges by searching for zero-crossings.
ZeroCross	Filters the image with a size the user specifies within the function, next it find the edges by looking for zero-crossings.
Canny	This method calculates the gradient using the derivative of Gaussian filter. It employs two thresholds; one to detect weak edges and the other to detect strong edges. This leads to more accurate results that aren't disrupted by noise.

3.2.3 Code B – Registering

This piece of code followed the steps written in the design above. The different types of registration are all shown and the difference between them are shown in the results Chapter 3.3.3 below.

3.2.4 Code C – Reconstruction

In this part of the project, the radon and inverse radon technique did not support the type of images that was assigned to this project (Y-stack images). Since it mostly supports A-scans different angles from 0 to 179 (MathWorks, 2019). Consequently, the next technique to attempt was to stack the B-scans provided as slices above each other. Each image was a 2-D image and to make a 3-D model, the 2-D images were stored in a 3-D array where X, Y were the 2-D image attributes and Z was the number of the slice. Afterwards, the 3-D array was shown through the usage of function volshow() which displays a volume (MathWorks, 2019).

3.2.5 Code D – CNN

Within the process of this code, many complications were met. First step was to pre-process the images; this was done by going through the image pixel by pixel and checking if the pixel within the range of specified colour that Dr Peter have pointed out. Afterwards, it would be coloured red where then the percentage of red pixel within a box

size is higher than 60% it would be coloured green. That indicates that it has a high chance of decay. The first complication arose during pre-processing the images to be inputted into the CNN. There was not enough memory to pre-process all the images thus the code would skip a few pixels and not process them. After that, the CNN was built using keras and tensorflow as a backend library. After training the CNN with the processed images a second complication occurred. It would always detect the images given has decay. To overcome this complication, the CNN was trained using supervised learning as opposed to unsupervised. Thus, the images processed were labelled manually. However, the classifier maintained classifying the images as decayed. Due to the amount of time left, memory access limitations as well corona virus this was put aside for further work.

3.2.6 Web app

Through the development of this code, no complications were met. The GUI was shown in Chapter 3.3.6 below.

3.2.7 Software

While developing this code, there was one complication where it was not able to put before and after image on the same window. Thus, it was overcome by having a main window were the user chooses the function. When asked to upload the image, a Before window is shown with the image chosen. After the process has occurred, the result image would be shown on an After window as well as a message telling the user where the images were saved locally.

3.3 Results

3.3.1 Code A – Denoising

- Within this code, the best optimiser for OCT images was identified through the values of loss function and RMSE. The results are shown in Table 6; they conclude that the ADAM optimizer is the leading optimizer. Since both the loss function and RMSE is significantly lower than SDGM optimizer.

Table 6: Shows the results of loss and RMSE for each optimizer

Optimizer	Loss function	RMSE
ADAM	2.874	2.221
SDGM	6.518	4.335

- Subsequently, the image quality was tested after each filter it goes through by using PSNR, MSE and SSIM for each type of image provided. In addition, the CNN was tested using the loss function and RMSE. Table 7, Figure 9, 10, 11 are the results of a Y-stack image denoising. Therefore, it has been concluded that as the original image goes through filters the MSE increases till it reaches CNN (from 1.06E+03 to 138.64) and PSNR decreases. But stay the same when going through BM3D and imsharpen(). SSIM increases gradually till it reaches CNN and it increases largely from 0.0476 to 0.89. This means that the CNN restores the image details as well as removes noise efficiently.

Table 7: Shows the results obtained from denoising a Y-stack image

Filter (a-scan)	MSE	PSNR	SSIM
Thresholding	522.121	20.953	0.114
FLLP	636.779	20.091	0.089
BM3D	1.06E+03	17.871	0.048

BM3DSharp	1.06E+03	17.871	0.048
Imsharpen()	1.06E+03	17.872	0.048
CNN	138.6407	26.712	0.890

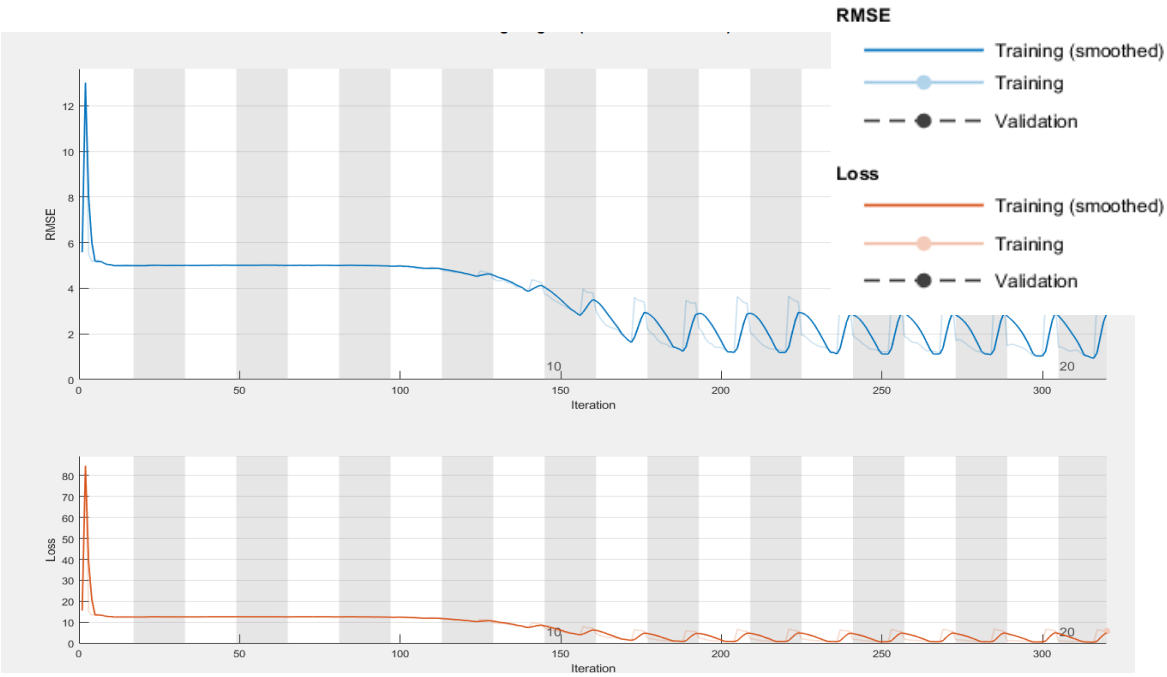

Figure 9: The graphs each display the descent of RMSE and loss as the CNN is training using training and validation sets of images. The legend on the top right dedicates the different lines on each graph. Each graph concludes that the RMSE and loss function decreases over the period of time.

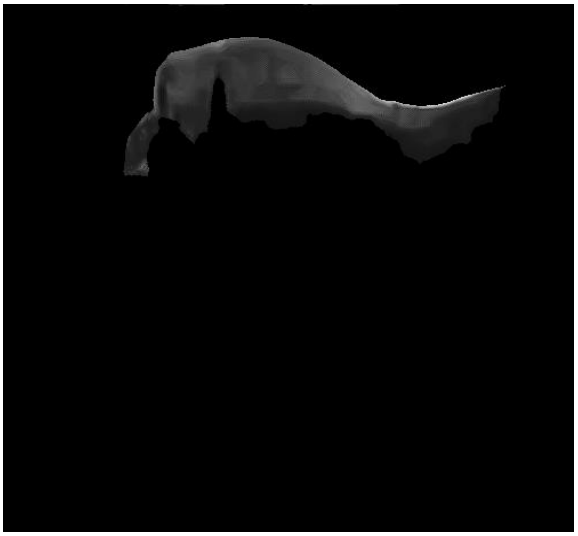

Figure 10: The Y-stack result image. It can be shown that it was fully denoised and the edges are accurately defined. This image was the output after Figure 11 was inputted into Code A in the appendix

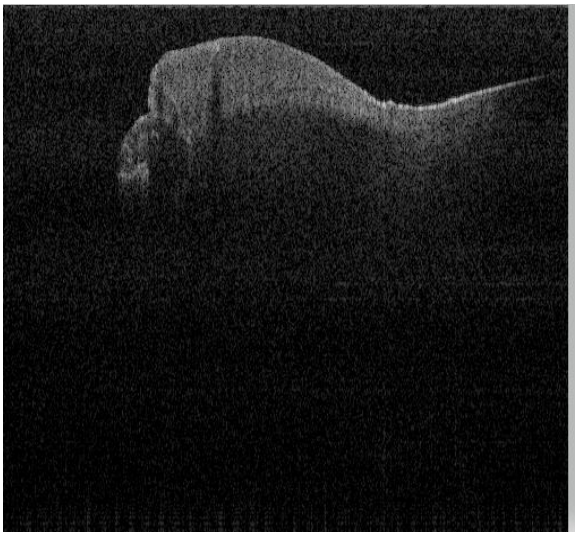

Figure 11: Y-stack image before denoising. Displays that the noise was speckle noise and there was a lot of it.

- The next step involved testing a B-scan of a tooth. These results are shown in Table 8, Figure 12, 13. Therefore it can be concluded, that every filter had no effect on the image except CNN. Due to the MSE, PSNR and SSIM staying relatively the same. Also, it is apparent that thresholding wasn't a success with this type of image, since the background was still a shade of grey instead of black.

Table 8: Shows the numerical results of denoising a B-scan

Filter (B-scan)	MSE	PSNR	SSIM
Thresholding	1.46E+08	14.669	0.072
FLLP	1.46E+08	14.696	0.075
BM3D	1.47E+08	14.669	0.068
BM3DSharp	1.47E+08	14.669	0.068
Imsharpen()	1.47E+08	14.669	0.068
CNN	8.86E+05	36.853	0.896

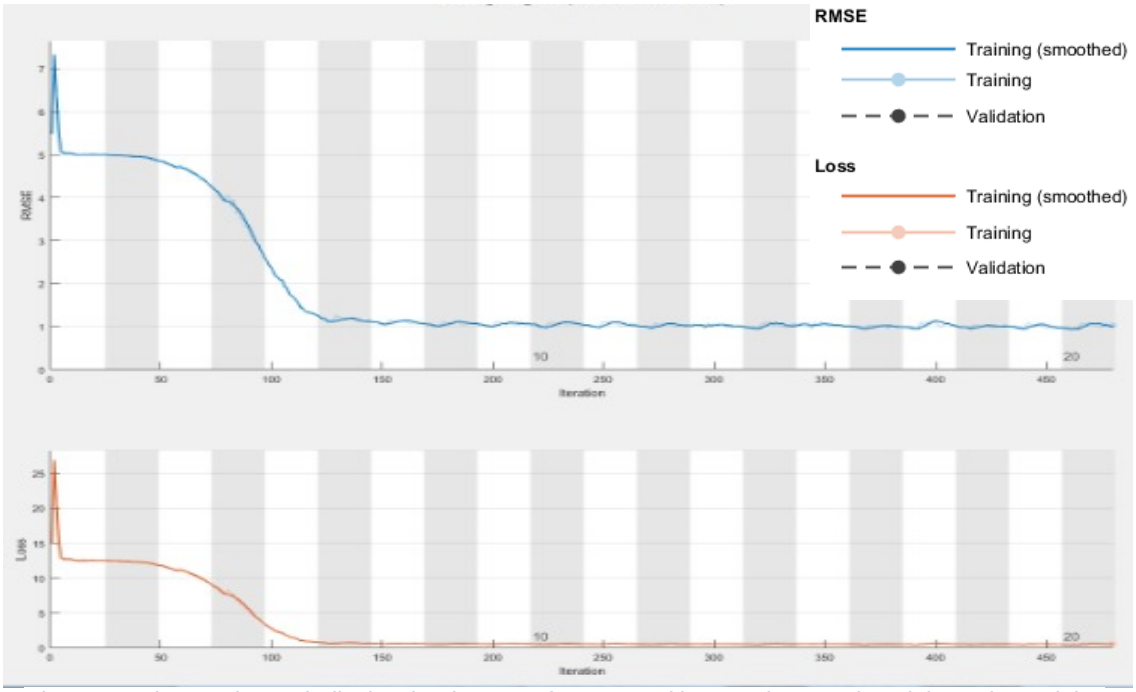

Figure 12: The graphs each display the descent of RMSE and loss as the CNN is training using training and validation sets of images. The legend on the top right dedicates the different lines on each graph. Each graph concludes that the RMSE and loss function decreases over the period of time

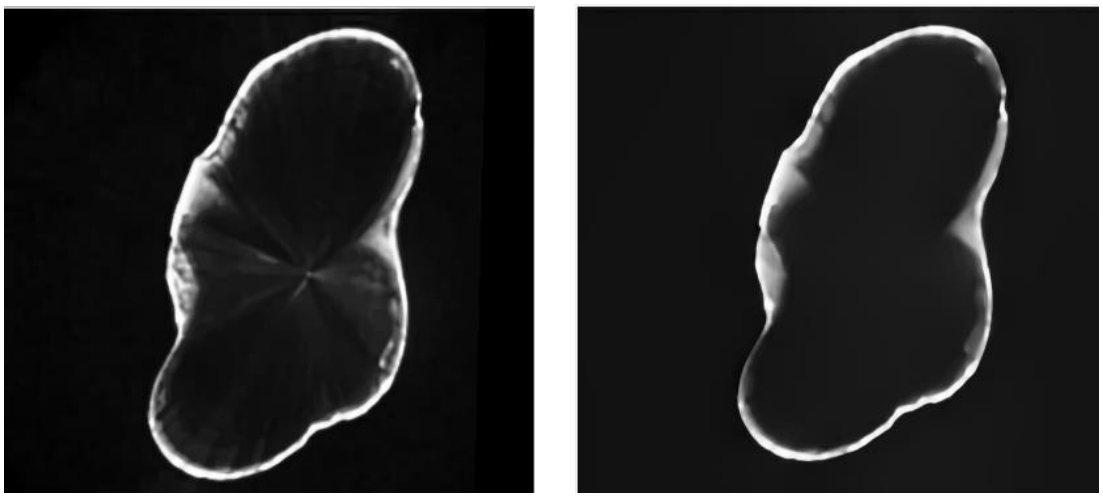

Figure 13: Displays the before (left) and after (right) image of B-scan denoising. The before image included noise as streaks of white inside the borders of the B-scan. It can be concluded that the noise inside the borders have been removed efficiently. The after image was the result of inputting the before image into Code A in the appendix.

- Afterwards, an image of a synthetic tooth with brackets is tested. The results are shown in Table 9, Figure 14, 15, 16. It can be concluded that BM3D and imsharpen() increase the MSE value (1.53E+03 to 2.12E+03) but decrease the PSNR (16.27 to 14.86). However, CNN drops the MSE largely from 2.12E+03 to

160.51. Regarding the CNN, as it learns the loss and RMSE decrease due to it learning more features. As well as, a restoring of the image features through the massive increase in SSIM from 0.0476 to 0.8540. Therefore, it can be said that CNN is relatively better filter.

Table 9:Results of denoising synthetic tooth with brackets

Filter (brackets)	MSE	PSNR	SSIM
Thresholding	1.53E+03	16.273	0.065
FLLP	1.71E+03	15.803	0.089
BM3D	2.12E+03	14.864	0.052
BM3DSharp	2.12E+03	14.864	0.052
Imsharpen()	2.12E+03	14.864	0.048
CNN	160.5173	26.076	0.854

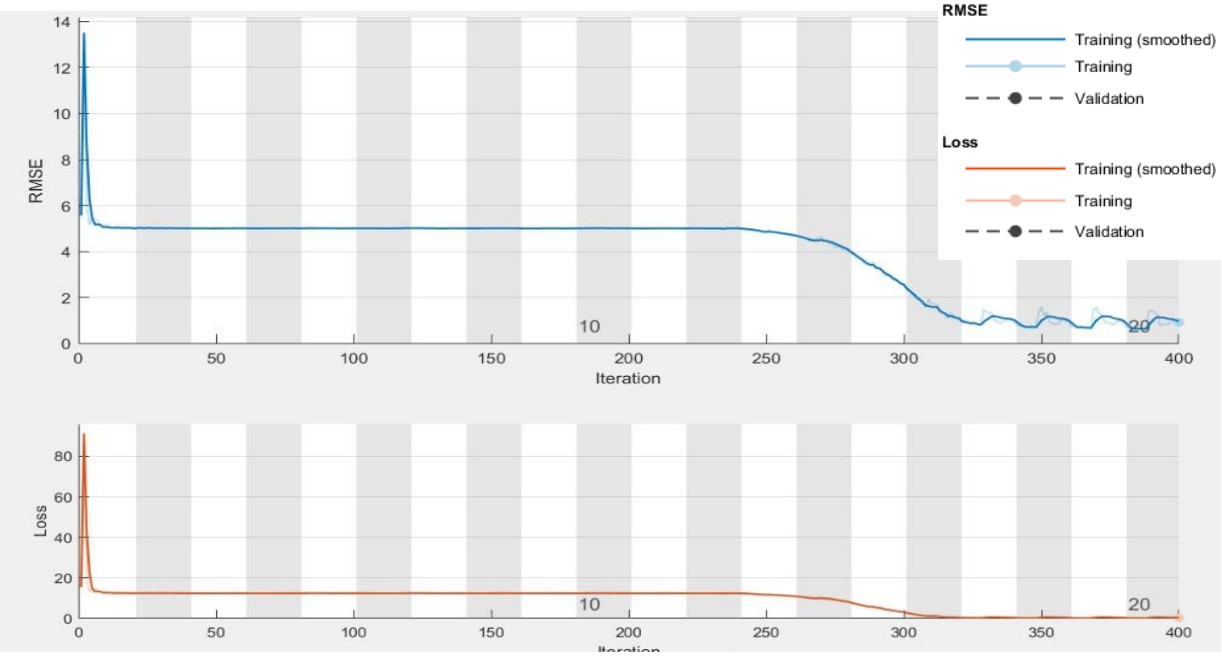

Figure 14: The graphs each display the descent of RMSE and loss as the CNN is training using training and validation sets of synthetic tooth images. The legend on the top right dedicates the different lines on each graph. Each graph concludes that the RMSE and loss function decreases over the period of time. This occurs as the code goes on and thus the CNN learns new features hence decreasing loss function and RMSE.

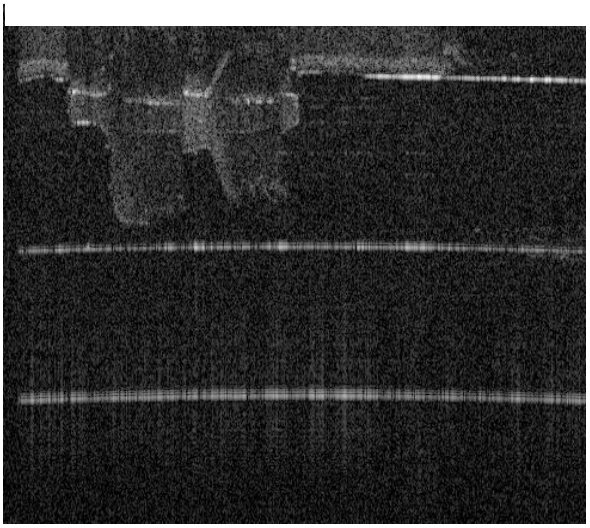

Figure 15: The original image of synthetic teeth with brackets before denoising. It displays the noise within the image, as white streaks and grey backgrounds.

Figure 16: The result of the brackets image after denoising. This shows that the image was successfully thresholded, as well as the edges were well defined.

3.3.2 Code A – Masking

Masking CNN was used for the brackets image as it had the most significance. The results are shown below. As shown in Figure 17, the masking CNN learns the edges of the image and further thresholds the image. In conclusion, Figure 18 shows that the important outline can be clearly seen as opposed to the denoised image (Figure 19).

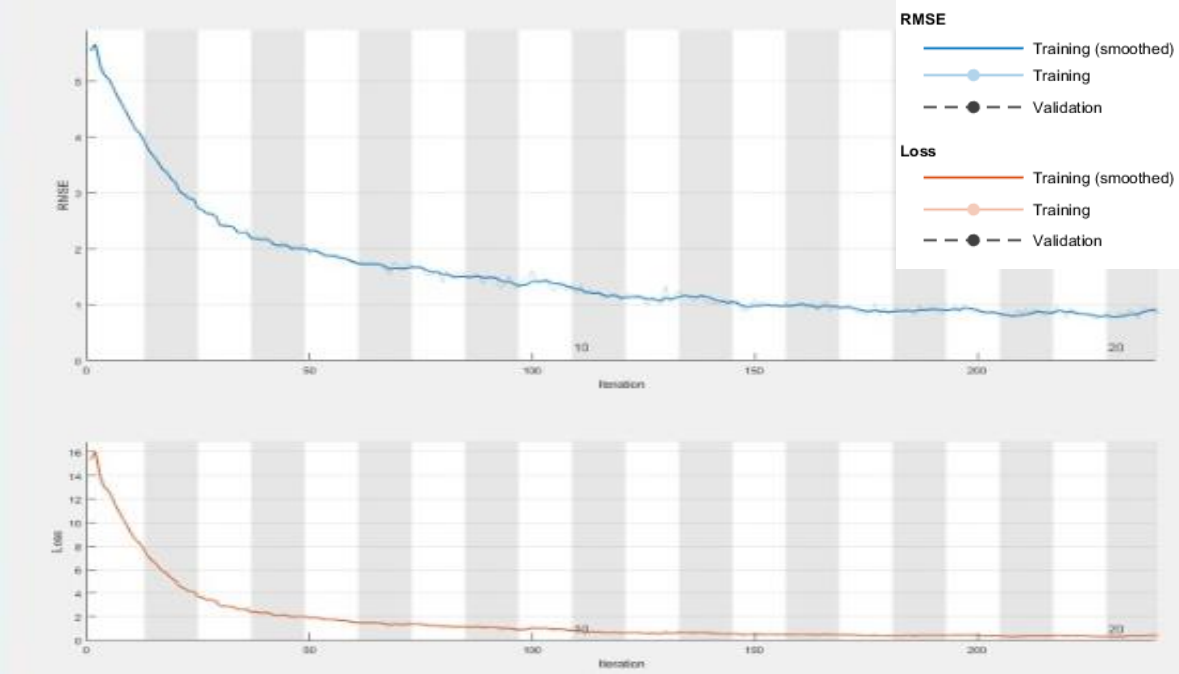

Figure 17: The graphs each display the descent of RMSE and loss as the CNN is training using training and validation for synthetic teeth with brackets for the masking code. The legend on the top right dedicates the different lines on each graph. Each graph concludes that the RMSE and loss function decreases over the period of time.

Figure 18: The resulting image of masking CNN, it shows that the edges are more defined through drawing a white line over the real edges and blacking the background. (after masking CNN).

Figure 19: This is the imagethat goes into the masking CNN. It has already been denoised using code A in appendix. (before masking CNN)

3.3.3 Code B – Registering

Testing this piece of code was through visualisation. Firstly, Monomodal registration was tested. The moving and fixed images were given by Dr Peter, where the OCT image were registered to an XMT image. The moving image was the OCT, fixed image was the XMT. The result is shown in Figure 20. Next, Multimodal registration was tested. The

results are shown in Figure 21. Regarding both figures, it can be concluded that multimodal has better results than modal (Looking at the translation registration on both figures 20,21). Since it attempts to register all the edges onto each other, but modal attempts to register the first edge it notices. Afterwards, the types of registration are

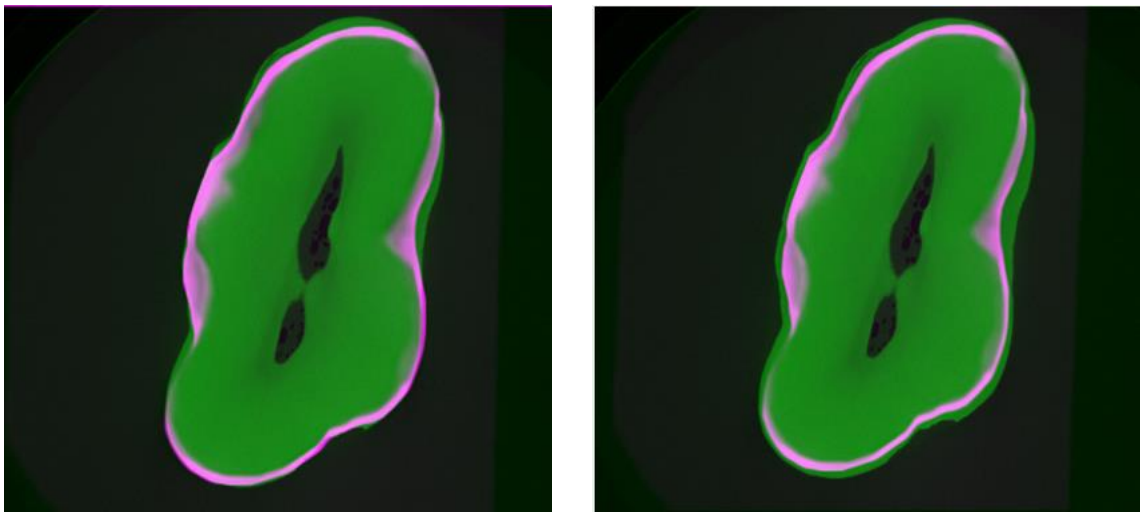

Figure 20: Shows the results of using Monomodal registration, where green is the XMT image and purple is OCT. Left- Before. Right- After translation registration. It is shown that the edges have not been aligned correctly throughout the whole border.

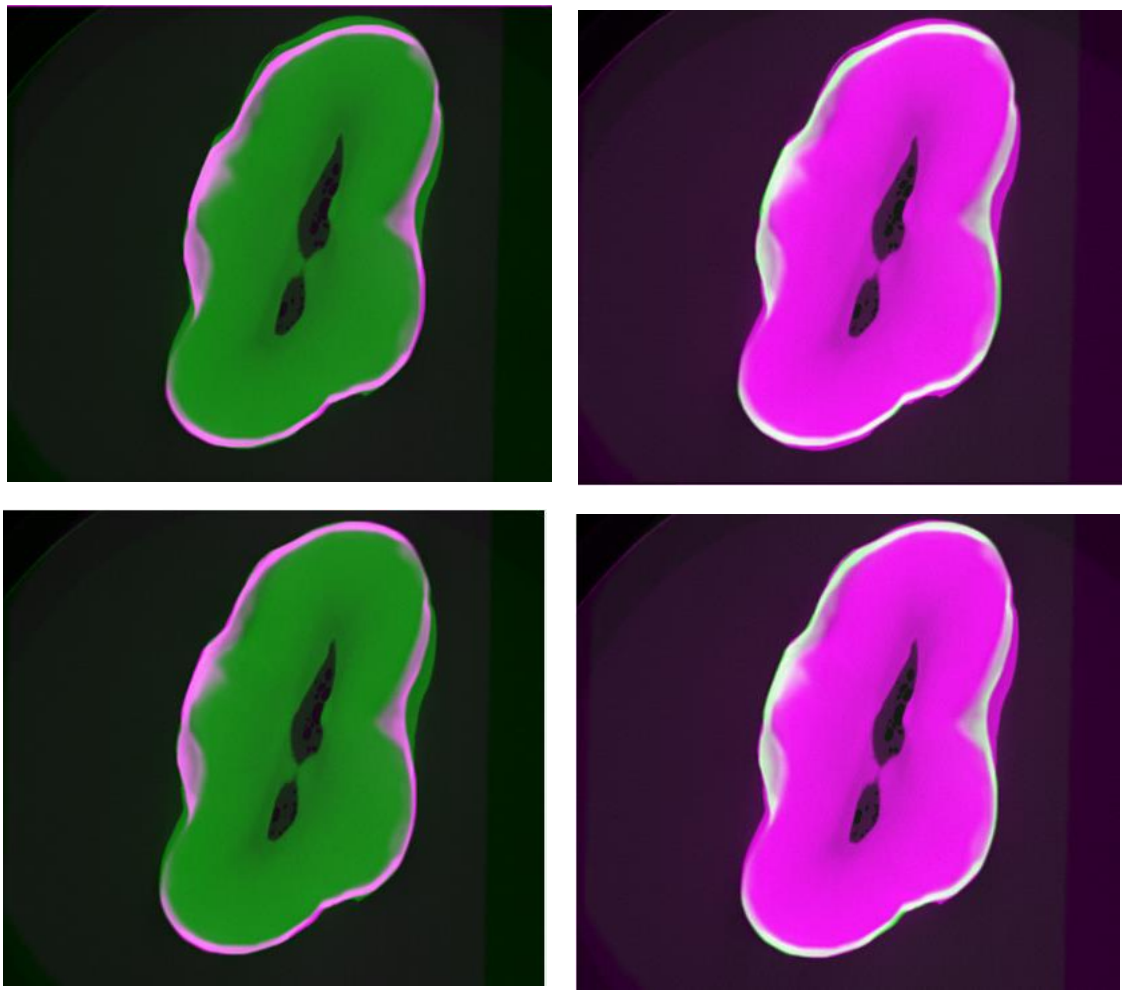

Figure 21: This shows the 4 results from Multimodal were green is the XMT image and purple is OCT. Top Left- Before Image. Top Right-Result using Rigid registration. Bottom Left- Result using Translation registration. Bottom Right- Result using Affine Registration. The outcome of these show that the Affine registration obtained the most successful registering of all the edges relatively accurate. This used code B.

compared. It can be concluded that affine registration was superior compared to other techniques due to the usage of shear. This enables it to register the whole edges fittingly.

3.3.4 Code C – Reconstruction

Testing this code involved inputting the file path of where the B-scans are stored. Then the code will arrange the images into a 3D array. Through the usage of imsubtract() different values are attempted. Outcomes are shown in Figures 22,23,24. It was further

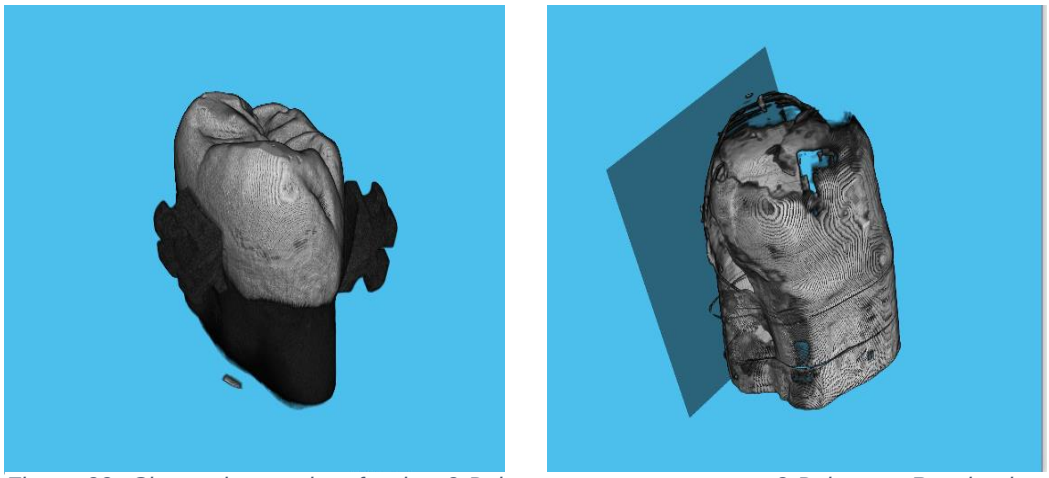

Figure 22: Shows the results of using 2-D images to reconstruct a 3-D image. Result when imsubtract function is set to 30. XMT reconstruction is on the left and OCT is on the right. It shows that the top of the tooth is fully removed. This used code C.

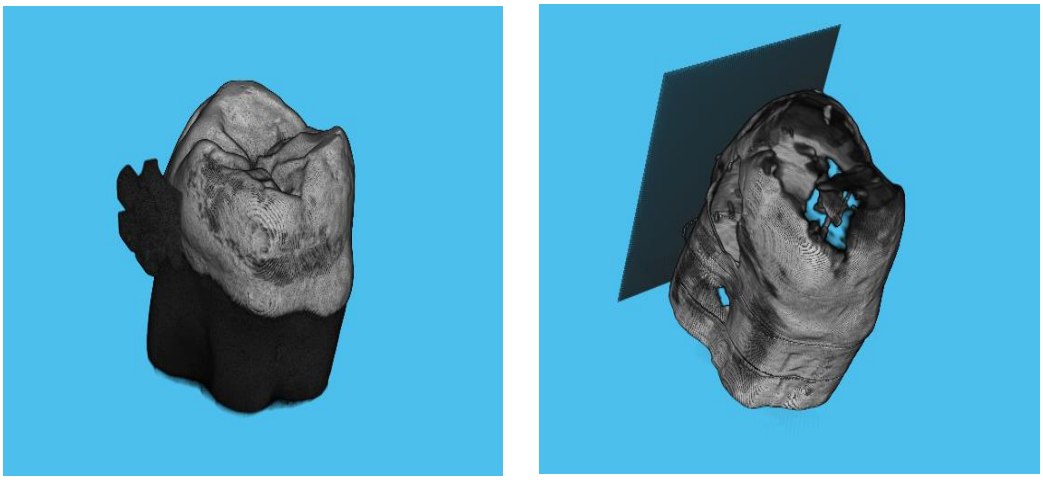

Figure 23: Result when function imsubtract is set to 20. XMT reconstruction is on the left and OCT is on the right. It shows that the top of the tooth is shady and empty.

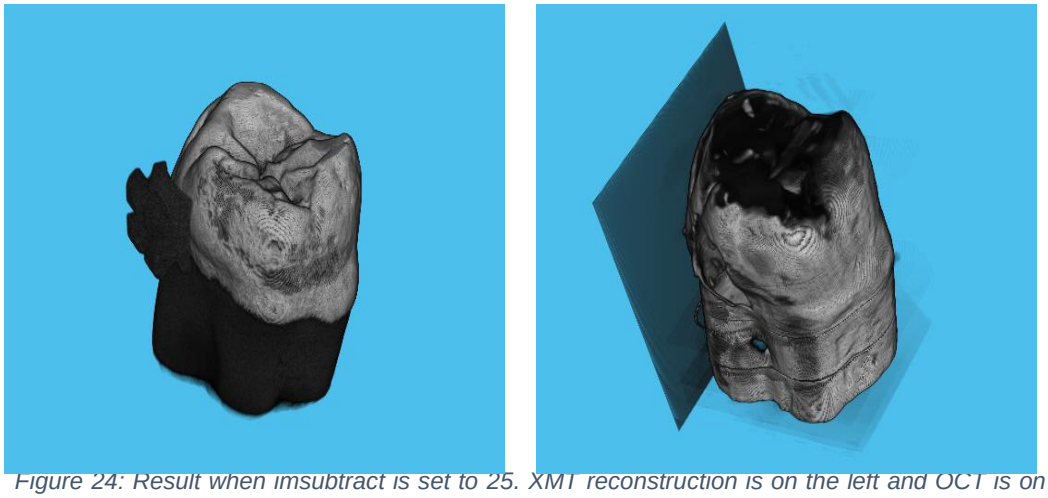

Figure 24: Result when imsubtract is set to 25. XMT reconstruction is on the left and OCT is on the right. It shows that the top of the tooth is not defined and blurred out.

concluded that the OCT images that were provided had no registrations for the top part of the tooth. Thus there are no details to be shown.

3.3.5 Code D – CNN

Within this code, the only results obtained were the pre-processed images. It shows the suggested decay very clearly and relatively accurate (Figure 25). However, the CNN always predicted that any image put through had decay.

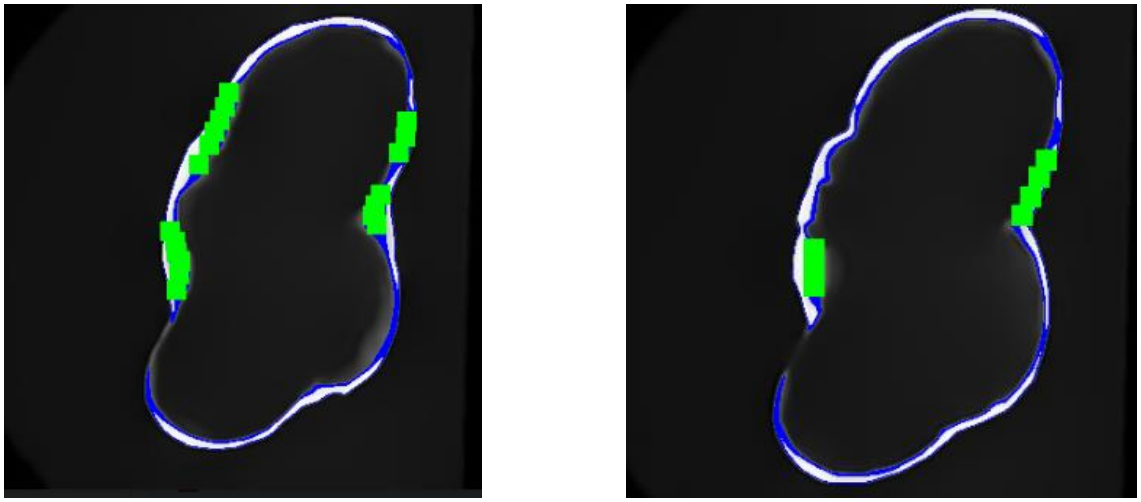

Figure 25: Displays two images after image pre-processing. The blue shading is the first stage of predicting decay; the green shading is the second stage of accurately predicting decay.

3.3.6 Web app

In this section, the web app produced will be shown. It includes a top navigation bar, where it incorporates all the functions produced from the previous code mentioned (Figure 26, 27). Every page has a different function, thus it requires various inputs from the user. At each function, there is a bottom navigation bar where it explains what will occur during their wait for the results as well as a short explanation what each function does.

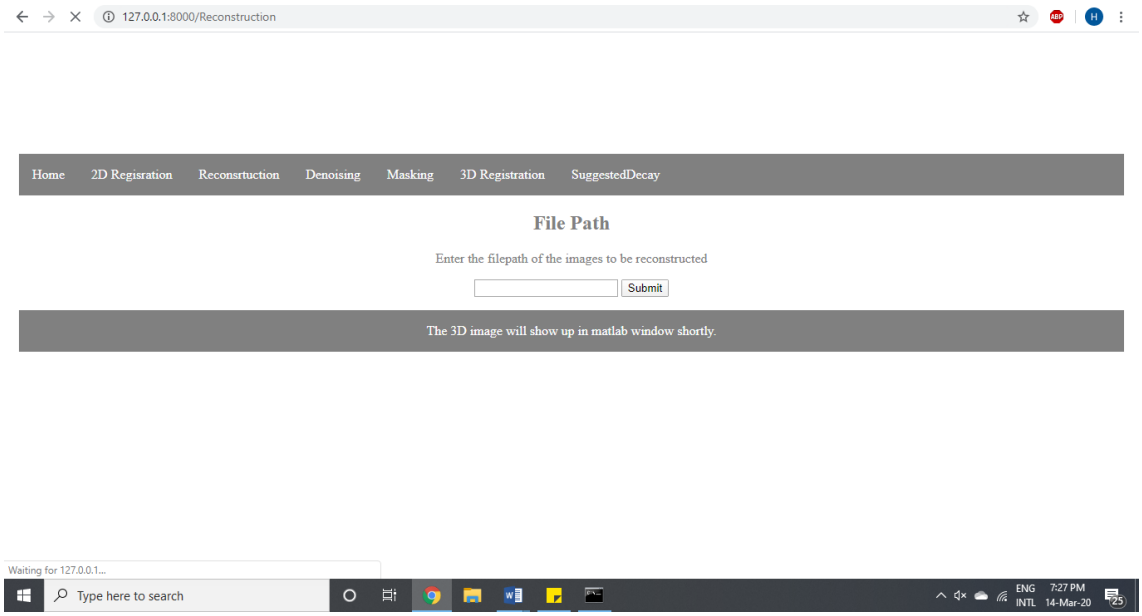

Figure 26: Displays the website user interface when Reconstruction tab is chosen. This window will be displayed to the user when Webapp code in the appendix is run through Django in python in the command line.

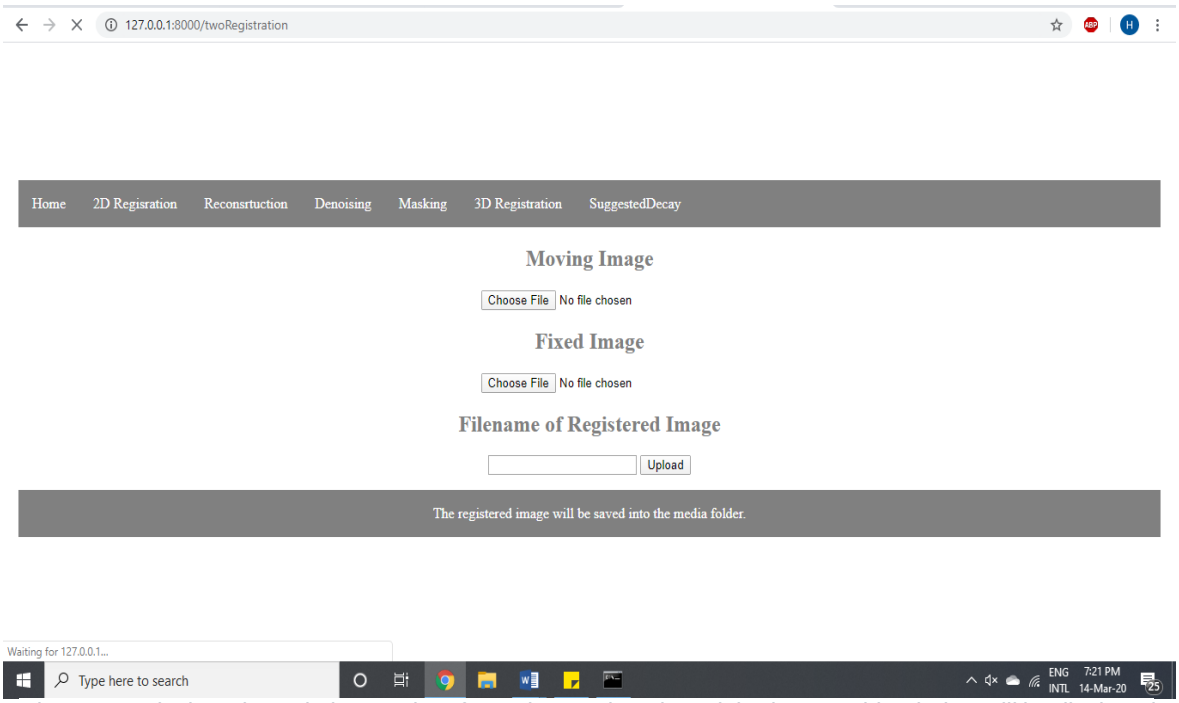

Figure 27: Displays the website user interface when registration tab is chosen. This window will be displayed to the user when Webapp code in the appendix is run through Django in python in the command line.

Another feature that was added through using the django forums was the control of the users entering the local host. It was controlled through the admin page of the website (Figure 28, 29).

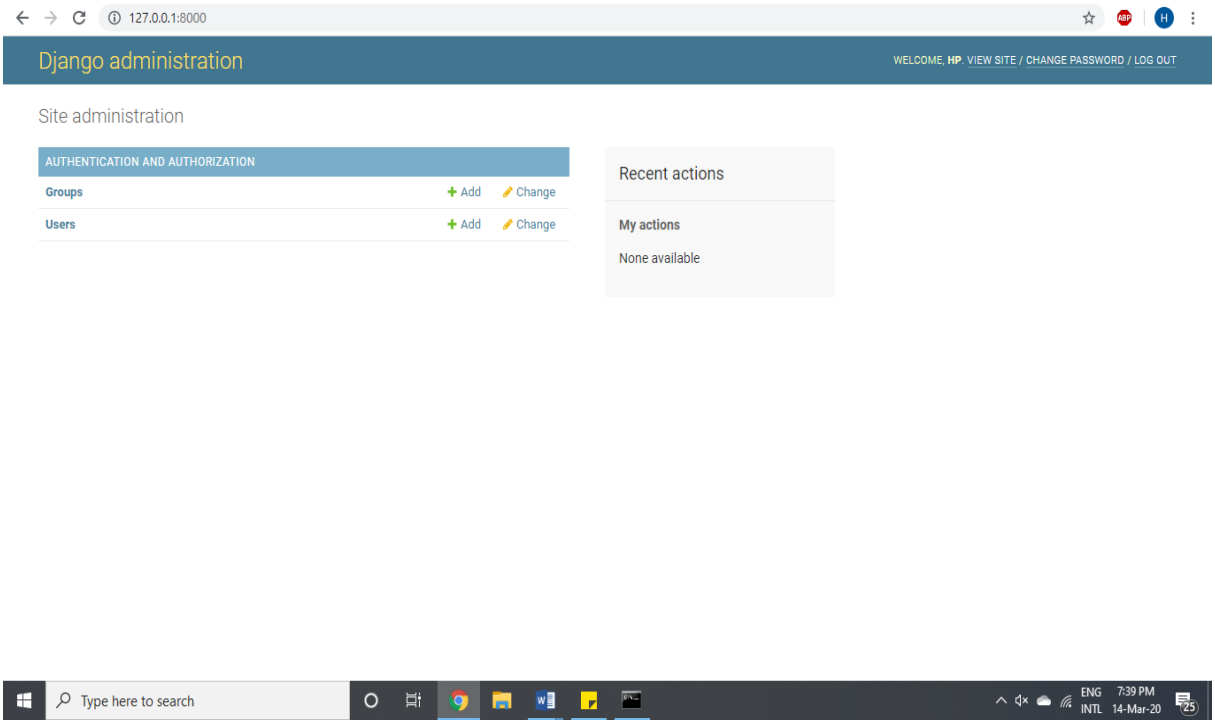

Figure 28: This shows the admin site page provided by Django through python. Where the admin can add users to be able to access the website privately. This window will be displayed to the user when Webapp code in the appendix is run through Django in python in the command line.

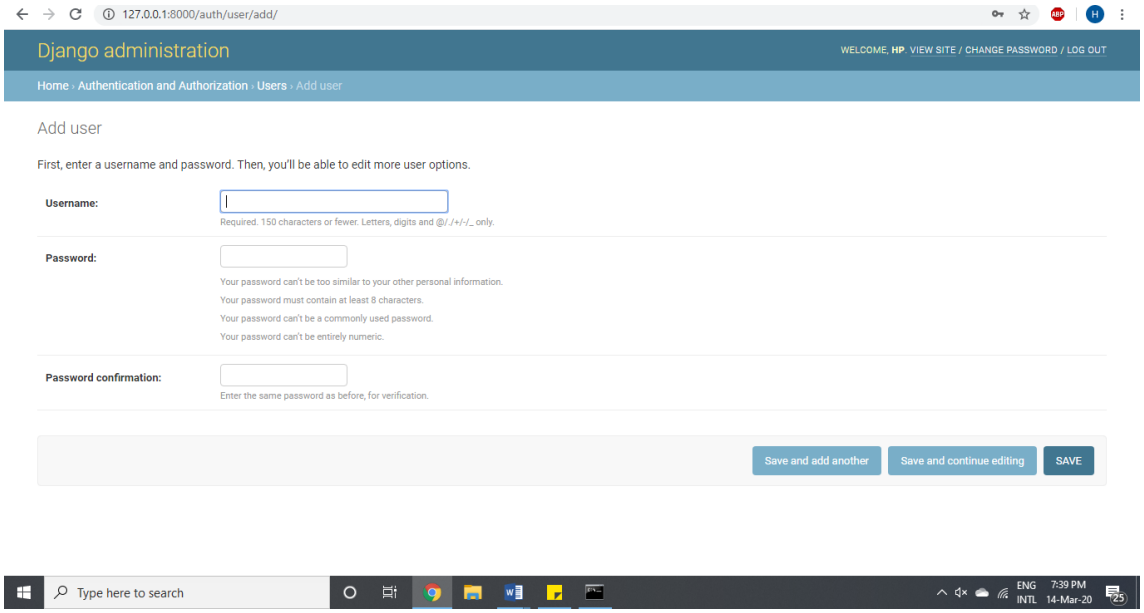

Figure 29: Shows the forum page where a username and password is set up by the main local host. This window will be displayed to the user when Webapp code in the appendix D is run through Django in python in the command line.

3.3.7 Software

In this section, the software produced will be shown. Similar to the web app, it includes a top navigation bar showing the user the different functions it provides (Figure 30). In addition, each function has a small description for what occurs during the process and where the images will be saved locally.

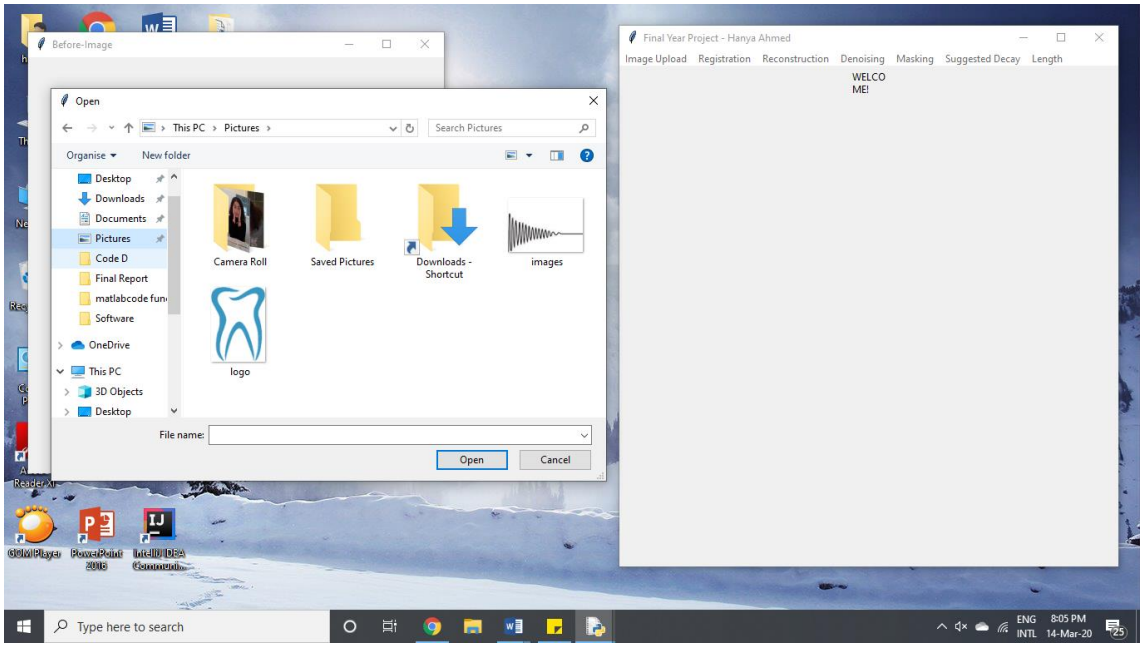

Figure 30: Displays the Main window (right) and Before window (left). The before window opened when "Image Upload" is clicked. This allows the user to upload whichever image on the current desktop available. These windows will open when the Software code in the appendix is run through Tkinter in python in the command line. The code used for this is in appendix E.

3.4 Evaluation

The aims of this project were to firstly denoise images, register OCT to XMT images, reconstruct a 3D model from 2D images and to detect tooth decay. Within each aim, a number of experiments were conducted to arrive to the leading process.

Several denoising filters were experimented with including BM3D, FLLP, CNN and thresholding. Several error calculations and image types were used to conclude that CNN had the least MSE and highest SSIM when compared to BM3D, FLLP and thresholding.). Thus, it removed most noise as well as restored each image. However, BM3D had the lowest PSNR at each type of image which means that BM3D outperforms CNN in the signal to noise ratio.

Through numerous error calculations and different types of images (B-scan, Y-stack, synthetic tooth), it was concluded that CNN had the least MSE (138, 8.86E+05, 160) and highest SSIM (0.89, 0.89,0.85) and PSNR (26.7, 36.8, 26.1). Thus, it has removed most noise as well as restored each image. However, BM3D had the lowest PSNR at each type of image which means that BM3D outperforms CNN in the signal to noise ratio.

Subsequently, one CNN was used to denoise and mask the image. However, this was found tedious and so an attempt was done to use two separate CNNs and connecting them together. This way one CNN will be used to denoise and the other to mask the image, where the second CNN was fed the output of the first CNN that goes through edge detection. This was deduced more appropriate by comparing the images produced and their RMSE and loss function.

To ensure optimal usage of the CNN, two optimizers were tested; ADAM and SGDM. They were tested through the loss function and RMSE. ADAM had 2.87 and 2.22 respectively, while SDGM has 6.5 and 4.33 respectively. Indicating that the ADAM optimizer learnt more features in images than SDGM as well as lost relatively less information. Thus, ADAM was the leading optimizer.

For the analysis of the several techniques of registering the OCT to XMT images, it was summarised that Multimodal affine registration was the leading option for registration. This was deduced due to its ability to overlap most of the edges on both images relatively absolute.

Once the images were denoised, masked and registered they needed reconstruction. Radon theorem was not able to support the B-scans provided and thus could not achieve the objective. Therefore, the technique attempted was to stack the images into a 3D array. This was successful and it will aid clinicians to visualize the full tooth. In addition, it might show the patient the tooth decay more efficiently.

The detection of decay within the OCT image was not successful in this project. However, the utilization of CNNs is heavily recommended. Due to the time constraint, COVID-19 and software constraints this project wasn’t able to produce results for the detection of decay.

Furthermore, a Webapp and software were produced. It was deduced that GUI (Graphical User Interface) helps externals understand the code better and the function occurring. It aids clinicians and further researchers to utilize the codes in a straightforward manner as well as effortlessly.

Chapter 4: **Conclusion**

This project aimed to have a fully denoised 3D image from multiple 2D images, to help detect tooth decay through machine learning. This was important, since according to National Health and Nutrition Examination Survey there are 20.5% of children aged between 2-5 years old have tooth decay and 24.5% of children between 6-11 years old have tooth decay (NHANES, 2018). As well as nearly all early tooth decay is undetected until it advances to a pit or a cavity.

Within this project, the 2D image was denoised as well as registered to the XMT file to give a clearer view to clinicians. This improved the process because it combined the features of both images, XMT and OCT. Also, the multiple 2D images were reconstructed to give a perspective to dentists as well as researchers. Along with the website and software they provide a helpful GUI to explain the process of what is occurring and the results that will show.

It has been concluded that CNN is indeed the leading approach to denoise and mask images appropriately through the numerical results and the resulting images. As well as, that the ADAM optimizer trains the CNN to achieve the best results. Also, the usage of affine registration thoroughly matches the features on both image as much as possible, shown through multiple images. Using the images as slices which fit quite well is the optimal method to reconstruct B-scan images into a 3D model.

In conclusion this project worked towards helping OCT image more useable in the dental field, through many techniques and to help detect tooth decay. Further work has been proposed in the Chapter 4.1.

4.1 Further Work

- Possible further work would be regarding Code D, where the CNN would be able to predict correctly. As well as reconstruct a 3D model from that CNN and produce a heat map across the 3D image to show how severe the tooth decay is.
- Another possibility is through using a more functional computer where it will provide a better RAM access as well as multiple CPUs rather than a single CPU.
- Regarding the website and software, additional GUI programming would make it easier for a broader audience to use easily.
- Also, broadening the codes to be able to denoise, reconstruct, register, analyse all types of OCT images, and not only dental images.

References

American Dental Association, 2019. *Decay.* [Online] Available at: https://www.mouthhealthy.org/en/az-topics/d/decay

Biomedical Optics Research Group (BORG), 2011. *Simon Fraser University.* [Online] Available at: http://borg.ensc.sfu.ca/fourieroct.html [Accessed November 2019].

Brian Krans, K. C., 2018. *Dental X-Rays.* [Online] Available at: https://www.healthline.com/health/dental-x-rays [Accessed October 2019].

Brownlee, J., 2016. *Supervised and Unsupervised Machine Learning Algorithms.* [Online]
Available at: https://machinelearningmastery.com/supervised-and-unsupervised-machine-learning-algorithms/
[Accessed November 2019].

Callaham, J., 2019. *The hisory of Anroid OS: Its name, origin and more.* [Online] Available at: https://www.androidauthority.com/history-android-os-name-789433/ [Accessed November 2019].

Chen, Y. a. X. H., 2013. Comparative Study of Speckle Noise Reduction Approaches for Interferometric Synthetic. *Applied Mechanics and Materials,* Volume 427, pp. 1735-1738.

Chen, Y. a. X. H., 2013. ComparativeStudy of Speckle Noise Reduction Approaches for Interferometric Synthetic Aperture Radar Images.. *Applied Mechanics and Materials,* Volume 427-429, pp. 1735-1738.

Chinn SR, S. E. F. J., 1997. Optical Coherence tomography using frequency-turnable optical source. *JG Opt Lett.,* Volume 22(5), pp. 340-342.

Chong, B. a. Z. Y., 2013. Speckle reduction in optical coherence tomography images of human finger skin by wavelet modified BM3D filter. *Optics Communications,,* Volume 291, pp. 461-469.

Colston B. W., S. U. S. D. L. B. E. M. J. 6., 1998. Dental OCT Optical Express. p. 3.

Colston B.W., S. U. J. D. L. E. M. S. P. O. L., 1998. Dental OCT. *Opt. Express.,* Volume 3, pp. 230-238.

Dan P. Popescu, c. a. L.-P. C.-S. C. F. Y. M. S. C. J. D. S. S. a. M. G. S., 2011. Optical coherence tomography: fundamental principles, instrumental designs and biomedical applications. August.

Davis, D. T., 2017. *Different Types of Dental Imaging.* [Online] Available at: https://www.buckheadgadentist.com/2017/05/different-types-dental-imaging/
[Accessed October 2019].

Demirturk Kocasarac H, A. C., 2018. Ultrasound in Dentistry: Toward a Future of Radiation-Free Imaging.. *Dent Clin North Am,* 62(3), pp. 481-489.

Deshpande, A., 2017. *A Beginner's Guide To Understanding Convolutional Neural Networks.* [Online] Available at: https://www.commonlounge.com/discussion/515df1b9a20c48c68a09b28389709219 [Accessed November 2019].

Django Software Foundation, 2020. *Django.* [Online] Available at: https://www.djangoproject.com/ [Accessed 12 March 2020].

Dove, J., 2019. *Digital Trends.* [Online] Available at: https://www.digitaltrends.com/mobile/best-travel-apps/ [Accessed November 2019].

F. Heide, M. S. Y.-T. T. M. R. D. P. D. R. O. G. J. L. W. H. K. E. e. a. F., 2014. A flexible camera image processing framework.. *ACM Transactions on Graphics,* 33(6), p. 231.

FDI, 2019. *FDI World Dental Foundation.* [Online] Available at: https://www.fdiworlddental.org/oral-health/ask-the-dentist/facts-figures-and-stats [Accessed November 2019].

Feldchtein F., G. V. I. R. e. a., 1998. In vivo OCT imaging of hard and soft tissue of the oral cavity. *Optics Express,* Volume 14, p. 239–250.

Filho, J. C. B. L., 2013. Optical coherence tomography for debonding evaluation: An in-vitro qualitative study. *AJO-DO,* 143(1), pp. 61-68.

Frederick A Wilson, L. L. T., 1975. Characterization of the passice and active transport mechanisms for bile acid uptake into rat isolated intestinal epithelial cells. *Biochimica et Biophysica Acta (BBA)-Biomembranes ,* 406(2), pp. 280-293.

Fritzke, B., 1994. Growing Cell structures - a self-organizing network for unsupervised and supervised learning. *Neural Networks,* Volume 7.9, pp. 1441-1460.

Fu, B. X. Y. L. Y. X. H. a. Y. G., 2019. A Convolutional Neural Networks Denoising Approach for Salt and Pepper Noise. *CoRR.*

GeeksforGeeks, 2019. *MVC Design Pattern.* [Online] Available at: https://www.geeksforgeeks.org/mvc-design-pattern/ [Accessed November 2019].

GoodpasterBH, C. V. e. a., 2001. Attenuation of skeletal muscle and strength in the elderly: the Health ABC Study. *J Appl Physiol,* Volume 90, p. 2157–2165.

Hasan, M., 2019. Adaptive Edge-guided Block-matching and 3D filtering (BM3D) Image Denoising Algorithm.. *Electronic Thesis and Dissertation Repository,* Volume 2003.

Holst, A., 2019. Global market share smartphone operating systems of unit shipments 2014-2023. *Technology & Telecommunications.*

Holtzman J. S., O. K. P. J. e. a., 2010. Ability of optical coherence tomography to detect caries beneath commonly used dental sealants. *Lasers in Surgery and Medicine,* Volume 42, p. 752–759.

Huang D., S. E. L. C. S. J. S. W. C. W. H. M. F. T. G. K. P. C. F. J., 1991. Optical Coherence Tomography. *Science,* Volume 254, pp. 1178-1181.

J.Clement, 2019. Google Play: number of availabe apps 2009-2019. *Internet.*

K. Dabov, A. F. V. K. a. K. E., 2007. Image denoising by sparse 3-D transform-domain collaborative filtering. *IEEE Transactions on Image Processing,* 16(8), pp. 2080-2095.

Kai Zhang, W. Z. Y. C. D. M. a. L. Z., 2016. Beyond a Gaussian Denoiser: Residual Learning of. pp. 1-4.

Karch, M., 2019. *What is Google Play?.* [Online] Available at: https://www.lifewire.com/what-is-google-play-1616720 [Accessed November 2019].

Karimian, N. et al., 2018. Deep learning classifier with optical coherence tomography images for early dental caries detection. *SPIE. Digital Library*, February.

Karimian, N. et al., 2018. Deep learning classifier with optical coherence tomography images for early dental caries detection. *SPIE Digital Library* , Feburary.

Karn, U., 2016. *An Intuitive Explanation of Convolutional Neural Networks.* [Online] Available at: https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/ [Accessed October 2019].

Katkovnik., K. E. a. V., 2015. Single image superresolution via BM3D sparse coding. *European Signal Processing Conference ,* pp. 2849-2853.

Larry Gearce, R. F., 1994. Integral membrane protiens and dynamic orginization of the nuclear envelope. *Trends in cell biology,* 4(4), pp. 127-131.

Lebrun, M., 2012. An Analysis and Implementation of the BM3D Image Denoising Method. *Image Processing On Line,* Volume 2, pp. 175-213.

Lin, J. W.-B., 2012. Why Python Is the Next Wave in Earth Sciences Computing. *Physics Department, North Park University, Chicago, Illinois,* Volume 93, p. 1823–1824.

Lotz, M., 2018. Waterfall vs. Agile: Which is the Right Development Methodology for Your Project?. 5 July .

Maia A. M., F. D. D. K. B. B. G. A. S., 2010. Characterization of enamel in primary teeth by optical coherence tomography for assessment of dental caries. *International Journal of Paediatric Dentistry,* Volume 20, p. 158–164.

Make Use Of, 2019. *"Is Anroid Really Open Source? And Does It Even Matter".* [Online] Available at: https://www.makeuseof.com/tag/android-really-open-source-matter/ [Accessed November 2019].

MathWorks , 2019. *Image Registration.* [Online] Available at: https://uk.mathworks.com/discovery/image-registration.html [Accessed November 2019].

MathWorks United Kingdom, 2019. *Specify Layers of Convolution Neural Networks.* [Online]
Available at: https://uk.mathworks.com/help/deeplearning/ug/layers-of-a-convolutional-neuralnetwork.html

MathWorks, 2018. *Matrix Indexing.* [Online] Available at: https://uk.mathworks.com/company/newsletters/articles/matrix-indexing-in-

matlab.html
[Accessed October 2019].

MathWorks, 2018. *Vectorization.* [Online] Available at: https://uk.mathworks.com/help/matlab/matlab_prog/vectorization.html [Accessed October 2019].

MathWorks, 2019. *Approaches to registering images.* [Online] Available at: https://uk.mathworks.com/help/images/approaches-to-registering-images.html
[Accessed November 2019].

MathWorks, 2019. *Create an Optimizer and Metric for Intensity-Based Image Registration.* [Online] Available at: https://uk.mathworks.com/help/images/create-an-optimizer-and-metric-for-intensity-based-image-registration.html
[Accessed 13 March 2020].

MathWorks, 2019. *Edge.* [Online] Available at: https://uk.mathworks.com/help/images/ref/edge.html [Accessed 13 March 2020].

MathWorks, 2019. *Image.* [Online] Available at: https://uk.mathworks.com/profucts/image.html [Accessed Novemeber 2019].

MathWorks, 2019. *Image based automatic image registration.* [Online] Available at: https://uk.mathworks.com/help/images/intensity-based-automatic-image-registration.html
[Accessed November 2019].

MathWorks, 2019. *Image Reconstruction.* [Online] Available at: https://uk.mathworks.com/discovery/image-reconstruction.html [Accessed November 2019].

Mathworks, 2019. *imregister.* [Online] Available at: https://uk.mathworks.com/help/images/ref/imregister.html [Accessed March 2020].

MathWorks, 2019. *VolShow.* [Online] Available at: https://uk.mathworks.com/help/images/ref/volshow.html [Accessed 13 March 2020].

McAndrew, A., n.d. *An Introduction to Digital Image with MATLAB.* 1 ed. s.l.:s.n.

Michaela Goodwin, C. S., 2018. Issues arising following a referral and subsequent wait for extraction under general anaesthetic: impact on children. *BMC Oral Health,* 15(3).

Monika Machoy, 1. ,. *. J. S. 2. L. S.-S. R. K. T. G. a. K. W., 2017. The Use of Optical Coherence Tomography in Dental Diagnostics: A State-of-the-Art Review. *Journal of Healthcare Engineering,* Volume 2017.

Monika Machoy, J. S. L. S.-S. R. K., 2017. The Use of Optical Coherence Tomography in Dental. *Journal of Healthcare Engineering,* Volume 2017, p. 31.

Morris, C., 2010. Boston Tech-Mafia Mondays: TripAdvisor – Pioneers in Online Travel. *Bostinno.*

Nan Zang, M. B. R. V. N., 2008. Mashups: who? what? why?. *Extended Abstracts on Human Factors in Computing Systems,* pp. 3171-3176.

Navedtra, 1999. *Dental Technician Volume 1.* s.l.:s.n.

NHANES, 2018. *Dental Caries in Children.* [Online] Available at: https://www.nidcr.nih.gov/research/data-statistics/dental-caries/children

NOVACAM, 2019. *How LCI works.* [Online] Available at: https://www.novacam.com/technology/how-lci-works/ [Accessed November 2019].

Ohanian, T., 2017. 10 Usability Heuristics for User Interfaces in Web Design. *Medium*, 5 December.

Oral-B, 2019. *Tooth Decay and Cavities: Symptoms, Causes and treatment.* [Online] Available at: https://oralb.com/en-us/oral-health/conditions/cavities-tooth-decay/tooth-decay-cavities-symptoms-causes-treatments

Otis L. L., A.-S. R. I. M. J. B. R., 2003. Identification of occlusal sealants using optical coherence tomography. *The Journal of Clinical Dentistry,* 14(1), pp. 7-10.

Otis L. L., C. B. W. J. E. M. J. N. H., 2000. Dental optical coherence tomography: a comparison of two in vitro systems. *Dento Maxillo Facial Radiology,* Volume 29, p. 85–89.

Pagnoni A., K. A. W. P. R. M. S. T. K. L. S. I. K. A., 1999. Optical coherence tomography in dermatology.. *Skin Res. Technol.,* Volume 5, pp. 83-87.

Paris, S. H. S. K. J., 2011. Local Laplacian Filters: Edge-aware Image Processing with a Laplacian Pyramid. *ACM Trans,* 30(4), p. 11.

Perez, S., 2016. TripAdvisor scoops up social mapping service Citymaps. *TechCrunch*, 25 August.

Poneros J.M., B. S. B. B. T. G. C. C. N. N., 2001. Diagnosis of specialized intestinal metaplasia by optical coherence tomography. *Gastroenterology,* Volume 120, p. 7–12.

Prabhu, 2018. *Understanding of Convolutional Neural Network (CNN) — Deep Learning.* [Online]
Available at: https://medium.com/@RaghavPrabhu/understanding-of-convolutional-neural-network-cnn-deep-learning-99760835f148
[Accessed October 2019].

Python, 2019. *Tkinter.* [Online] Available at: https://wiki.python.org/moin/TkInter [Accessed 12 March 2020].

QiDou, H. L. L. J. V. C. D. M. L. S. a. P.-A., 2016. Automatic detection of cerebral microbleeds from MR images via 3D convolutional neural networks. *IEEE Trans. Med. Imaging,* 35(5), p. 1182–1195.

Renieblas, G. N. A. G. A. G.-L. N. a. d. C. E., 2017. Structural similarity index family for image quality assessment in radiological images. *Journal of Medical Imaging,* 4(3), pp. 1-7.

Roger G Christianson, K. M. F., 1999. Comparsion of student learning about diffusion and osmosis in constructivist. *International Journal of Science Education,* 21(6), pp. 687-698.

Ruder, S., 2016. *An overview of gradient descent optimization algorithms. CoRR.* [Online]
Available at: http://arxiv.org/abs/1609.04747
[Accessed November 2019].

S.Pereira, A. V. a. C. A., 2016. Brain tumor segmentation using convolutional neural networks in MRI images. *IEEE Trans Med. Imaging ,* 35(5), pp. 1240-1251.

Saxena, N. a. R. N., 2013. Speckle Reduction in SAR images using Wavelet Transform and Bivariate Shrinkage Function. International Journal of Advanced Research in Computer Science and Electronics Engineering (IJARCSEE). Volume 2, pp. 248 - 251.

Schmidt-Nielsen, K., 1984. *Scaling Why is animal size so important?.* s.l.:Cambridge University Press.

Schmitt, J. X. S. a. Y. K., 1999. Speckle in Optical Coherence Tomography. *Journal of Biomedical Optics ,* 4(1), p. 95.

Shayla K Banerji, M. A. H., 2007. Examination of nonendocytotic bulk transport of nanoparticles across phospholipod membranes. *Langmuir,* 23(6), pp. 3305-3313.

Stefan Broer, P. A., 2002. Adaptation of plasma membrane amino acid transport mechanisms to physiological demands. *Eur J Physiol,* Volume 444, p. 457.

Study.com, 2020. *What is Translation in Math.* [Online]
Available at: https://study.com/academy/lesson/what-is-translation-in-math-definition-examples-terms.html
[Accessed 12 March 2020].

Su-Yeon Hwang, 1. E.-S. C. Y.-S. K. B.-E. G. M. H. a. H.-Y. K., 2018. Health Effects from exposure to dental diagnostic X-ray. *Environ Health Toxicol*, November.

Tanzila Saba, A. R. a. G. S., 2010. *AN INTELLIGENT APPROACH TO IMAGE DENOISING,* s.l.: JATIT & LLS.

Team Counterpoint, 2019. *Counterpointresearch.* [Online]
Available at: https://www.counterpointresearch.com/global-smartphone-share/
[Accessed November 2019].

U., H., 1978. Transport in isolated plasma membranes. *Am J Physiol,* 234(2), pp. 89-96.

U.S Department of Health and Human Services, 2014. *Dental Statistics (US & WorldWide).* [Online]
Available at: https://www.electricteeth.com/dental-statistics/

W. Dong, L. Z. G. S. a. X. L., 2013. Nonlocally centralized sparse representation for image restoration. *IEEE Transactions on Image Processing,* 22(4), pp. 1620-1630.

Wang Y., B. B. I. J. T. O. H. D., 2008. Retinalblood flow measurement by circumpapillary Fourier domain Doppler optical coherence tomography. *J. Biomed. Opt.,* p. 13.

Wilkinson, S., 2017. *Dental Jargon: What are They Talking About.* [Online] Available at: https://www.churchfielddental.co.uk/dental-jargon [Accessed October 2019].

Witten, I. H., 2017. *Data Mining.* 4th ed. s.l.:Todd Green.

Wong, A. M. A. B. K. a. C. D., 2010. General Bayesian estimation for speckle noise reduction in. *Optics Express,* 18(8), p. 8338.

World Health Organization, 2016. *Oral Health.* [Online] Available at: https://www.who.int/news-room/fact-sheets/detail/oral-health

Yeagle, P. L., 1989. Lipid Regulation of cell membrane structure and function.

Ymir Mäkinen, L. A., 2014. *Image and video denoising by sparse 3D transform-domain collaborative filtering.* [Online] Available at: http://www.cs.tut.fi/~foi/GCF-BM3D/ [Accessed November 2019].

Yu-Chi Lai, J.-Y. L. ,.-Y. Y. ,.-Y. L. ,.-Y. L. ,.-W. C. a. I.-Y. C., 2019. Interactive OCT-Based Tooth Scan and Reconstruction. *Sensors*, September.

Zhang, W., 1988. Shift-invariant pattern recognition neural network and its optical architecture. *Proceedings of Annual Conference of the Japan Society of Applied Physics.*

Zhang, W., 1991. Error Back Propagation with Minimum-Entropy Weights: A Technique for Better Generalization of 2-D Shift-Invariant NNs. *Proceedings of the International Joint Conference on Neural Networks.*

Zhang, W., 1994. Computerized detection of clustered microcalcifications in digital mammograms using a shift-invariant artificial neural network. *Medical Physics,* 24(4), pp. 517-24.

Appendix A – Code A

```
%This is a script for Code A Denoising
%This script will run CreateTData.m to the image you've inputted, to create
%training data. Then it will run TOCNN.m to creat and train the CNN. Then
%it will ask you to enter a filepath of the image again to output the
%denoised image.


%make sure to addpath the folder that contains the training images
addpath('TrainingDataCreated');


%prompt to ask the user for the filepath of the image to create training
%data
Input = input('Please enter the filepath of your image','s');


%trains the network using past images
%to denoise regular images then uncomment the line below
%TData = CreateTData(Input);
%to denoise registered images then uncomment the line below
TData = CreateTDataReg(Input);
%to denoise brackets images then uncomment the line below
%TData = CreateTDataBrackets(Input);
%to denoise Humanteeth images then uncomment the line below
%TData = CreateTDataHumanTeeth(Input);


%This trains the CNN with the newest training data
octDenoisingCNN = TOCNN('TrainingDataCreated');


% asks the user to input image for denoising
prompt ='Please enter the name of your image, i.e "Image1.png"';
Image = input(prompt, 's');


% inputing and changing the image into gray scale
Testimage = imread(Image);


%loading the perviously trained CNN
load TCNN;
load Training;


%denoises the image using the trained CNN
Testing = denoiseImage(Testimage,Training);
%displays the before and after images
imshowpair(Testing,Testimage,'montage');
%saves the after image in workspace
imwrite(Testing, 'DenoisingOutput.png','png');


%%ERROR CALCULATIONS
%uncomment below to print the errors
```

```
%    %CNN
%    CNNmse = immse(uint8(Testing), Testimage);
%    CNNpsnr = psnr(uint8(Testing), Testimage);
%    CNNssim = ssim(uint8(Testing), Testimage);
%    disp('CNN MSE');
%    disp(CNNmse);
%    disp('CNN PSNR');
%    disp(CNNpsnr);
%    disp('CNN SSIM');
%    disp(CNNssim);


%This is a script for Code A Masking
%make sure to addpath the folder that contains the training images
addpath('MaskingTD');


%add the images that have been produced by edges and the pervious CNN
%to train the next CNN
octMaskingCNN = MOCNN('MaskingTD');


% asks the user to input image for denoising
prompt ='Please enter the name of your image, i.e "Image1.pn"';
Image = input(prompt, 's');


% inputing and changing the image into gray scale
Testimage = imread(Image);


%loading the perviously trained CNN
load MCNN;
load MTraining;


%denoises the image using the trained CNN
Testing = denoiseImage(Testimage,MTraining);
%displays the before and after images
imshowpair(Testing,Testimage,'montage');
%saves the after image in workspace
imwrite(Testing, 'MaskingOutput.png','png');


%%ERROR CALCULATIONS
%uncomment below to print the errors
%    %CNN
%    CNNmse = immse(uint8(Testing), Testimage);
%    CNNpsnr = psnr(uint8(Testing), Testimage);
%    CNNssim = ssim(uint8(Testing), Testimage);
%    disp('CNN MSE');
%    disp(CNNmse);
%    disp('CNN PSNR');
%    disp(CNNpsnr);
%    disp('CNN SSIM');
%    disp(CNNssim);
```

Appendix B – Code B

```
%This script is for Code B- Image Registration
%prompts the user for the file path of the moving image
Moving = input('Please enter the filepath of the Moving image i.e Image.png',
's');
%It reads the png image or file path the moving image
MI = imread(Moving);
%it calculates the number of dimensions in the array
dimensionsImg = ndims(MI);
%if the no. of dimensions is larger than 2 then trun it into gray scale
if dimensionsImg > 2
    MI = rgb2gray(MI);
end


%prompts the user for the file path of the fixed image
Fixed = input('Please enter the filepath of the Fixed image i.e Image.png',
's');
%It reads the png image or file path the fixed image
FI = imread(Fixed);
%it calculates the number of dimensions in the array
dimensionsImg = ndims(FI);
%if the no. of dimensions is larger than 2 then trun it into gray scale
if dimensionsImg > 2
    FI = rgb2gray(FI);
end
%figure showing before
figure("Name", "Before");
imshowpair(FI, MI,'Scaling','joint'); %in purple green


% MONOMODAL REGISTRATION
%uncommnent below to use monomodal registration
% %optimizer for modal
% % optimizer = registration.optimizer.RegularStepGradientDescent();
% % metric = registration.metric.MeanSquares();
% %modifying the regular step gradient
% % optimizer.MaximumIterations = 300;
% % optimizer.MinimumStepLength = 5e-4;


% MULTIMODAL REGISTRATION
%optimizer and metric choosen for multimodal operation
optimizer = registration.optimizer.OnePlusOneEvolutionary();
metric = registration.metric.MattesMutualInformation();


%modifying the one plus one evolutionary
%we modifiy it so the transformation matrix is at its global maximum
optimizer.InitialRadius = 0.009;
optimizer.Epsilon = 1.5e-6;
optimizer.GrowthFactor = 1.01;
optimizer.MaximumIterations = 500;
optimizer.InitialRadius = optimizer.InitialRadius/4.5;
```

```
%MULTIMODAL Translation Registration
movingRegistered = imregister(MI,FI, 'affine', optimizer, metric);


%MULTIMODAL Similarity and Rigid Registration
%using imregtform to use the geometeric function
tformSimilarity = imregtform(MI,FI,'similarity',optimizer,metric);
Rfixed = imref2d(size(FI));
movingRegisteredRigid = imwarp(MI,tformSimilarity,'OutputView',Rfixed);


%MULTIMODAL Affine Registration
movingRegisteredAffineWithIC = imregister(MI,FI,'affine',optimizer,metric,...
'InitialTransformation',tformSimilarity);




%figures showing the after images
figure('Name', 'After- Translation Registration');
imshowpair(FI, movingRegistered,'Scaling','joint');
figure('Name', 'After- Rigid Registration');
imshowpair(movingRegisteredRigid, FI);
figure('Name', 'After- Affine Registration');
imshowpair(movingRegisteredAffineWithIC,FI);
```

Appendix C – Code C

```
%This script is Code C which is to construct the XMT and OCT registered
images


%make sure to addpaths to both folders containing the B-scans of XMT and OCT
addpath('Nok1_XMT_Reg');
addpath('OCTReg-denoised');


%XMT B-SCANS
%an unsigned int variable
numberXMT = uint8(0);
%for loop to generate all the images
%change the number depending on the number of images there are in the folder
for i=1: 419
    numberXMT = i;
    %read the image starting from the first one
    arrayXMT = imread(strcat('XMT_ (',sprintf('%d',numberXMT),').png'));
    %remove the background
    Z = imsubtract(arrayXMT,100);
    %insert the B-scan into a 3D array
    XMT3D(:,:,numberXMT) = Z;
end


figure('Name', 'XMT 3D');
%shows the 3D image
Model = volshow(XMT3D);


%OCT B-SCANS
%an unsigned int variable
numberOCT = uint8(0);
%for loop to generate all the images
%change the number depending on the number of images there are in the folder
for i=1: 419
    numberOCT = i;
    %read the image starting from the first one
    arrayOCT = imread(strcat('REOCT_ (',sprintf('%d',numberOCT),').png'));
    %remove the background
    Z1 = imsubtract(arrayOCT,20);
    %insert the B-scan into a 3D array
    OCT3D(:,:,numberOCT) = Z1;
end


figure('Name', 'OCT 3D');
%show the 3D image
Model1 = volshow(OCT3D);
```

Appendix D – Code D

```
#This script is for Code D - Detecting Decay
#However there are faults in this code that does not achieve the objective

#importing the needed libraries
import numpy as np
import pandas as pd
import cv2 as cv
from matplotlib import pyplot as plt

#Prompt for the user to enter the filepath of B-scan image
FilePath = input("Please Enter the filepath of the BScan i.e Image.png")

##XMT images pre processing

#reads the filepath
Im = cv.imread(FilePath)
#nested for loops to run through the image array to check if the pixel number
is between a certain color that corresponds to decay
#this is where the fault is, the pixel range is not the corrent color for
decay

for x in range(0, len(Im)-1):
  for y in range(0, len(Im[x])-1):
    if (sum(Im[x][y]) < 336 and sum(Im[x][y]) > 315):
      #if its within the range color it red
      Im[x][y] = [255,0,0]
    else:
      #else keep it as it is
      Im[x][y] = Im[x][y]

#saves the new image in the same folder
cv.imwrite("PreprocessingImage.png",Im)
#displays the image
plt.imshow(Im)
plt.show()

#XMT Green
#This part is further preprocessing where the suggested decay that is colored
red is checked through looking around the pixel
#and changing the pixel to green if neighboring pixels are also red
#reads the filepath

Im1 = cv.imread("PreprocessingImage.png")
#sets the box of neighboring pixels
boxsize = 15

#nested for loops to run through the image array
for x in range(len(Im1)-51):
  for y in range(len(Im1)-51):
    redpixels = 0
    #nested for loops to run through the box around selected pixel
    for i in range(boxsize):
      for j in range(boxsize):
        if np.array_equal(Im1[x+i][y+j], [255,0,0]):
          redpixels+=1
```

```
  if redpixels/(boxsize*boxsize) > 0.5:
    for b in range(boxsize):
      for c in range(boxsize):
        Im1[x+b][y+c] = [0, 255, 0]

#saves the new image in the same folder
cv.imwrite("GreenProcessedImage.png", Im1)
#displays the image
plt.imshow(Im1)
plt.show()
```

Appendix E – Software

```
#This is the script for simple setup for a software

#importing the needed libraries
from tkinter import *
from PIL.ImageTk import PhotoImage
from PIL import Image
from tkinter import filedialog
import numpy as np
import pandas as pd
import cv2 as cv
from matplotlib import pyplot as plt
import matlab.engine

#start matlab engine
eng = matlab.engine.start_matlab()

#opening a new window
window = Tk()
window.title("Final Year Project - Hanya Ahmed")
window.geometry('600x150')

#global variables
path = StringVar()
Mpath =StringVar()
Fpath = StringVar()
Bpath = StringVar()
HTpath = StringVar()
FTpath = StringVar()

##########################Functions#####################################
def uploadimage():
    #before window
    before = Tk()
    global path
    before.title("Before-Image")
    before.geometry('500x500')
    #asking the user to upload the image
    path = filedialog.askopenfilename()
    my_image = PhotoImage(file=path, master=before)
    c = Canvas(before, width=500, height=500)
    c.pack()
    c.create_image(0,0, image= my_image, anchor= NW)
    before.mainloop()

def uploadBimage():
    #before window
    beforeB = Tk()
    global Bpath
    beforeB.title("Brackets-Image")
    beforeB.geometry('500x500')
    #asking the user to upload the image
    Bpath = filedialog.askopenfilename()
    my_image = PhotoImage(file=Bpath, master=beforeB)
    c = Canvas(beforeB, width=500, height=500)
    c.pack()
```

```
    c.create_image(0,0, image= my_image, anchor= NW)
    beforeB.mainloop()

def uploadHTimage():
    #before window
    beforeHT = Tk()
    global HTpath
    beforeHT.title("Human-Teeth-Image")
    beforeHT.geometry('500x500')
    #asking the user to upload the image
    HTpath = filedialog.askopenfilename()
    my_image = PhotoImage(file=HTpath, master=beforeHT)
    c = Canvas(beforeHT, width=500, height=500)
    c.pack()
    c.create_image(0,0, image= my_image, anchor= NW)
    beforeHT.mainloop()

def uploadFTimage():
    #before window
    beforeFT = Tk()
    global FTpath
    beforeFT.title("Fake-Teeth-Image")
    beforeFT.geometry('500x500')
    FTpath = filedialog.askopenfilename()
    #asking the user to upload the image
    my_image = PhotoImage(file=FTpath, master=beforeFT)
    c = Canvas(beforeFT, width=500, height=500)
    c.pack()
    c.create_image(0,0, image= my_image, anchor= NW)
    beforeFT.mainloop()

def uploadMovingimage():
    #before window
    beforeM = Tk()
    global Mpath
    beforeM.title("MovingImage")
    beforeM.geometry('500x500')
    #asking the user to upload the image
    Mpath = filedialog.askopenfilename()
    my_image = PhotoImage(file=Mpath, master=beforeM)
    c = Canvas(beforeM, width=500, height=500)
    c.pack()
    c.create_image(0,0, image= my_image, anchor= NW)
    beforeM.mainloop()

def uploadFixedimage():
    #before window
    beforeF = Tk()
    global Fpath
    beforeF.title("FixedImage")
    beforeF.geometry('500x500')
    #asking the user to upload the image
    Fpath = filedialog.askopenfilename()
    my_image = PhotoImage(file=Fpath, master=beforeF)
    c = Canvas(beforeF, width=500, height=500)
    c.pack()
    c.create_image(0,0, image= my_image, anchor= NW)
    beforeF.mainloop()
```

```
def register():
    global Mpath
    global Fpath
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #starting matlab engine
    eng = matlab.engine.start_matlab()
    #running the matlab code
    Image1 = eng.ImageReg(Mpath,Fpath)
    #saving the output image
    cv.imwrite("RegisteredImage.png",Image1)
    #displays the output image in after window
    my_image = PhotoImage(file="PreProcessed.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The registering is done, you will find the
image saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def pre():
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('900x900')
    global path
    c = Canvas(after, width=400, height=400)
    c.pack()
    #This piece of code runs the python code for SuggestedDecay
    ##XMT images pre processing
    #reads the filepath
    Im = cv.imread(path)
    #nested for loops to run through the image array to check if the pixel
number is between a certain color that corresponds to decay
    #this is where the fault is, the pixel range is not the corrent color for
decay
    for x in range(0, len(Im)-1):
      for y in range(0, len(Im[x])-1):
        if (sum(Im[x][y]) < 336 and sum(Im[x][y]) > 315):
          #if its within the range color it red
          Im[x][y] = [255,0,0]
        else:
          #else keep it as it is
          Im[x][y] = Im[x][y]
    cv.imwrite("PreProcessed.png",Im)
    #displays the output image in after window
    my_image = PhotoImage(file="PreProcessed.png", master=after)
    c.create_image(0,0, image=my_image, anchor= NW)
    mes = Message(after, text="The first Pre Processing is done, you will
find the image saved in your local folder", anchor =SW)
    mes.pack()
    #XMT Green
```

```
    #This part is further preprocessing where the suggested decay that is
colored red is checked through looking around the pixel
    #and changing the pixel to green if neighboring pixels are also red
    #reads the filepath
    Im1 = cv.imread("PreProcessed.png")
    #sets the box of neighboring pixels
    boxsize = 15
    #nested for loops to run through the image array
    for x in range(len(Im1)-51):
      for y in range(len(Im1)-51):
        redpixels = 0
        #nested for loops to run through the box around selected pixel
        for i in range(boxsize):
          for j in range(boxsize):
            if np.array_equal(Im1[x+i][y+j], [255,0,0]):
              redpixels+=1
      if redpixels/(boxsize*boxsize) > 0.5:
        for b in range(boxsize):
          for c in range(boxsize):
            Im1[x+b][y+c] = [0, 255, 0]

    cv.imwrite("GreenProcessed.png", Im1)
    #displays the output image in after window
    my_image1 = PhotoImage(file="GreenProcessed.png", master=after)
    c.create_image(0,30, image=my_image1, anchor= NW)
    mes = Message(after, text="The Second Pre Processing is done, you will
find the image saved in your local folder", anchor =S)
    mes.pack()
    after.mainloop()

def denoisingB():
    global Bpath
    #after winodw
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image2 = eng.CreateTDataBrackets(Bpath)
    cv.imwrite("TrainingImage.png",Image2)
    #start the CNN
    Denoised = ImageDenoising('TrainingDataCreatedBrackets')
    #saves the output image
    cv.imwrite("DenoisedImage.png",Denoised)
    #displays the output image in after window
    my_image = PhotoImage(file="DenoisedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def denoisingHT():
    global HTpath
    #after window
    after = Tk()
```

```
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image2 = eng.CreateTDataHumanteeth(HTpath)
    cv.imwrite("HTTrainingImage.png",Image2)
    #start the CNN
    Denoised = ImageDenoising('TrainingDataCreatedHT')
    #saves the output image
    cv.imwrite("HTDenoisedImage.png",Denoised)
    #displays the output image in after window
    my_image = PhotoImage(file="HTDenoisedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def denoisingFT():
    global FTpath
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image2 = eng.CreateTData(FTpath)
    cv.imwrite("FTTrainingImage.png",Image2)
    #start the CNN
    Denoised = ImageDenoising('TrainingDataCreated')
    #saves output image
    cv.imwrite("FTDenoisedImage.png",Denoised)
    #displays the output image in after window
    my_image = PhotoImage(file="DenoisedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def denoisingR():
    global path
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image2 = eng.CreateTDataReg(path)
    cv.imwrite("FTTrainingImage.png",Image2)
    #start the CNN
    Denoised = ImageDenoising('TrainingDataCreatedReg')
    #saves output image
```

```
    cv.imwrite("FTDenoisedImage.png",Denoised)
    #displays the output image in after window
    my_image = PhotoImage(file="DenoisedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def maskingB():
    global Bpath
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image3 = eng.CreateTDataBrackets(Bpath)
    cv.imwrite("BTrainingImage.png",Image3)
    #start the CNN
    Masked = eng.ImageMasking('MaskingTDBrackets')
    #saves output image
    cv.imwrite("BMaskedImage.png",Masked)
    #displays the output image in after window
    my_image = PhotoImage(file="BMaskedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def maskingFT():
    global FTpath
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image3 = eng.CreateTData(FTpath)
    cv.imwrite("FTTrainingImage.png",Image3)
    #start the CNN
    Masked = eng.ImageMasking('MaskingTD')
    #saves output image
    cv.imwrite("FTMaskedImage.png",Masked)
    #displays the output image in after window
    my_image = PhotoImage(file="FTMaskedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def maskingHT():
```

```
    global HTpath
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image3 = eng.CreateTDataHumanteeth(HTpath)
    cv.imwrite("HTTrainingImage.png",Image3)
    #start the CNN
    Masked = eng.ImageMasking('MaskingTDHT')
    #saves output image
    cv.imwrite("HTMaskedImage.png",Masked)
    #displays the output image in after window
    my_image = PhotoImage(file="HTMaskedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def maskingR():
    global Rpath
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('250x250')
    mes = Message(after, text="MATLAB Figures will pop up during the
process", anchor =NW)
    mes.pack()
    #creating the training data
    Image3 = eng.CreateTDataReg(Rpath)
    cv.imwrite("RTrainingImage.png",Image3)
    #start the CNN
    Masked = eng.ImageMasking('MaskingTDReg')
    #saves output image
    cv.imwrite("RMaskedImage.png",Masked)
    #displays the output image in after window
    my_image = PhotoImage(file="RMaskedImage.png", master=after)
    c.create_image(0,2, image=my_image, anchor= NW)
    mes = Message(after, text="The process is done, you will find the image
saved in your local folder", anchor=SW)
    mes.pack()
    after.mainloop()

def rec():
    #rec window
    rec = Tk()
    rec.withdraw()
    #prompting the user to enter the file path and the number of images
    recFP = simpledialog.askstring(title="File name",
                                   prompt="Enter the filepath of the file of
images to be reconstructed")
    recnum = simpledialog.asksting(title="Number of images",
                                   prompt="Enter the number of images in the
file")
```

```
    #after window
    after = Tk()
    after.title("After-Image")
    after.geometry('600x600')
    mes = Message(after, text="MATLAB 3D model will pop up at the end of the
process", anchor =NW)
    mes.pack()
    #running Reconstruction
    Volume = eng.ImageReconstruction(Reconstruction,noimgs)
    mes = Message(after, text="The process is done, The output popped up at
the end of the matlab session", anchor=SW)
    mes.pack()
    after.mainloop()

########################################################################


#Menu Bar
menu = Menu(window)
#items
file = Menu(menu)
ctd = Menu(menu)
reg = Menu(menu)
den = Menu(menu)
mas = Menu(menu)
sug = Menu(menu)
rec = Menu(menu)

#FILE
file.add_command(label='Registered', command=uploadimage)
file.add_separator()
file.add_command(label='Brackets', command=uploadBimage)
file.add_separator()
file.add_command(label='Fake Teeth', command=uploadFTimage)
file.add_separator()
file.add_command(label='Human Teeth', command=uploadHTimage)
file.add_separator()
file.add_command(label='MovingImage', command=uploadMovingimage)
file.add_separator()
file.add_command(label='FixedImage', command=uploadFixedimage)
file.add_separator()
#Registration
reg.add_command(label='2D', command = register)
reg.add_separator()
#Denoising
den.add_command(label='Registered', command=denoisingR)
den.add_separator()
den.add_command(label='Brackets', command=denoisingB)
den.add_separator()
den.add_command(label='Fake Teeth', command=denoisingFT)
den.add_separator()
den.add_command(label='Human Teeth', command=denoisingHT)
den.add_separator()
#Masking
mas.add_command(label='Registered', command=maskingR)
mas.add_separator()
mas.add_command(label='Brackets', command=maskingB)
mas.add_separator()
```

```
mas.add_command(label='Fake Teeth', command=maskingFT)
mas.add_separator()
mas.add_command(label='Human Teeth', command=maskingHT)
mas.add_separator()
#Suggested Decay
sug.add_command(label='Decay',command=pre)
sug.add_separator()
#Reconstruction
rec.add_command(label='3D', command=rec)
#adding cascade
menu.add_cascade(label='Image Upload', menu=file)
menu.add_cascade(label='Registration',menu=reg)
menu.add_cascade(label='Reconstruction',menu=rec)
menu.add_cascade(label='Denoising',menu=den)
menu.add_cascade(label='Masking',menu=mas)
menu.add_cascade(label='Suggested Decay',menu=sug)


window.config(menu=menu)
window.mainloop()
```